

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ**



HCM-UTE

BÁO CÁO THỰC TẬP AI

BÀI 2: MACHINE LEARNING FOR REGRESSION

MÔN HỌC: THỰC HÀNH HỌC MÁY VÀ TRÍ TUỆ NHÂN TẠO

GIẢNG VIÊN HƯỚNG DẪN: PGS_TS. TRƯƠNG NGỌC SƠN

SINH VIÊN THỰC HIỆN: TRƯƠNG PHÚC THỊNH - 23161336

Tp. Hồ Chí Minh, tháng 5 năm 2026

MỤC LỤC

1. GIỚI THIỆU	1
1.1 Giới thiệu bài toán	1
1.2 Mục tiêu thực hiện	1
2. CƠ SỞ LÝ THUYẾT	2
2.1 Tổng quan về bài toán hồi quy	2
2.2 Linear Regression	2
Tìm hiểu về các thực hiện tìm đường hồi quy	4
2.3 Ridge Regression	8
2.4 Lasso Regression	9
2.5 Decision Tree Regressor	10
2.6 Random Forest Regressor	12
2.7 Các chỉ số đánh giá mô hình	14
2.7.1 Mean Absolute Error (MAE)	14
2.7.2 Root Mean Squared Error (RMSE)	14
2.7.3 R ² Score (Coefficient of Determination)	15
3. QUY TRÌNH THỰC HIỆN	16
3.1 Chuẩn bị và tải dữ liệu	16
3.2 Khám phá và phân tích dữ liệu	16
3.3 Xử lý dữ liệu thiếu và lựa chọn đặc trưng	17
3.4 Phân chia tập dữ liệu và huấn luyện mô hình	17
3.5 Tối ưu hóa và cải thiện mô hình	18
3.6 Môi trường và công cụ phát triển	18
4. KẾT QUẢ THỰC NGHIỆM	19
4.1 Các Bước Khởi Tạo Ban Đầu	19

4.1.1 Chuẩn bị môi trường.....	19
4.1.2 Tải dữ liệu.....	20
4.1.3 Kiểm tra thông tin tổng quan của dataset	20
4.2 Data Exploration (Bài tập 1).....	22
4.2.1 Kích thước của train và test dataset.....	22
4.2.2 Kiểm tra các cột có missing values	23
4.2.3 Tính tỷ lệ phần trăm missing values của từng cột.....	24
4.3 Visualization (Bài tập 2).....	25
4.3.1 Histogram của SalePrice.....	25
4.3.2 Scatter plot giữa GrLivArea và SalePrice	26
4.3.3 Scatter plot giữa OverallQual và SalePrice	28
4.4 Correlation Analysis (Bài tập 3).....	29
4.4.1 Tính Correlation Matrix	30
4.4.2 Vẽ Heatmap Correlation Matrix.....	31
4.4.3 Liệt kê 10 feature có tương quan cao nhất với SalePrice	33
4.5 Feature Selection (Bài tập 4)	34
4.6 Huấn Luyện Nhiều Mô Hình Regression (Bài tập 5).....	37
4.6.1 Linear Regression.....	37
4.6.3 Lasso Regression	40
4.6.4 Decision Tree Regressor.....	41
4.6.5 Random Forest Regressor.....	42
4.6.6 So sánh kết quả các mô hình	43
4.7 Model Evaluation – Đánh Giá Mô Hình (Bài tập 6)	47
4.7.1 MAE – Mean Absolute Error	47
4.7.2 RMSE – Root Mean Squared Error.....	47

4.7.3 R^2 Score	48
4.7.4 Bảng so sánh kết quả toàn bộ mô hình	48
4.8 Cross Validation (Bài tập 7)	50
4.8.1 Thực hiện 5-Fold Cross Validation	50
4.8.2 Tính giá trị trung bình.....	50
4.9 Hyperparameter Tuning (Bài tập 8)	51
4.10 Feature Engineering (Bài tập 9)	52
Huấn luyện lại mô hình với feature mới.....	52
4.11 Cải Thiện Mô Hình (Bài tập 10).....	54
4.11.1 Thử thêm feature	54
4.11.2 Thử thêm mô hình – Gradient Boosting Regressor.....	55
4.11.3 Điều chỉnh hyperparameters cho Random Forest	56
Tổng hợp kết quả cải thiện mô hình.....	58
5. PHÂN TÍCH VÀ THẢO LUẬN.....	59
5.1 Đánh giá tổng thể kết quả.....	59
5.2 Phân tích hiện tượng Overfitting và Underfitting	59
5.3 Ảnh hưởng của Hyperparameter Tuning	59
5.4 Hiệu quả của Feature Engineering	60
5.5 Lỗi gặp phải và cách khắc phục	60
5.6 Ưu điểm và hạn chế của các mô hình.....	61
6. KẾT LUẬN	63

MỤC LỤC HÌNH ẢNH

Hình 1: Minh họa thực tế bài toán hồi quy.....	3
Hình 2: Minh họa thực tế tiêu chuẩn đường hồi quy.....	4
Hình 3: Kết quả dịch chuyển đường hồi quy.....	6
Hình 4: Tất cả các đường hồi quy dịch chuyển đến khi hội tụ.....	7
Hình 5: Minh họa thực tế ridge regression.....	9
Hình 6: Minh họa thực tế random forest regression.....	13
Hình 7: Tổng quan về dữ liệu trên kaggle.....	20
Hình 8: Thông tin về dataset	22
Hình 9: Kích thước về dataset	23
Hình 10: Tổng hợp dữ liệu trong tập bị thiếu.....	23
Hình 11: Tỷ lệ missing values của từng cột	24
Hình 12: Biểu đồ về sự phân bố giá nhà	26
Hình 13: Biểu đồ mối quan hệ giữ diện tích sử dụng của căn nhà và giá bán	27
Hình 14: Biểu đồ mối quan hệ giữa chất lượng tổng thể và giá bán nhà	29
Hình 15: Bảng giá trị corelation matrix.....	31
Hình 15: Biểu đồ giá trị corelation matrix.....	32
Hình 16: Bảng liệt kê 10 feature có tương quan cao nhất với SalePrice.....	33
Hình 17: Kết quả các feature đầu vào	36
Hình 18: Kết quả hiển thị giá trị đầu tiên của biến SalePrice	37
Hình 19: Kết quả Accuracy của Linear Regression	38
Hình 20: Kết quả Accuracy của Ridge Regression	39
Hình 21: Kết quả Accuracy của Lasso Regression	40
Hình 22: Kết quả Accuracy của Decision Tree	42
Hình 23: Kết quả Accuracy của Random Forest.....	43

Hình 24: Kết quả Accuracy tổng hợp các mô hình	44
Hình 25: Các feature quan trọng trong random forest.....	46
Hình 26: Kết quả đánh giá chỉ số MAE Random Forest Regressor.....	47
Hình 27: Kết quả đánh giá chỉ số MRSE Random Forest Regressor.....	48
Hình 28: Kết quả đánh giá chỉ số R2 Core Random Forest Regressor	48
Hình 29: Bảng tổng hợp kết quả đánh giá mô hình.....	49
Hình 30: Kết quả 5-Fold Cross Validation của mô hình Random Forest	50
Hình 31: Kết quả giá trị trung bình 5-Fold Cross Validation của mô hình Random Forest	50
Hình 32: Kết quả tìm hyperparameter tối ưu bằng GridSearchCV	51
Hình 33: Kết quả tạo feature mới của AgeOfHouse và TotalArea	52
Hình 34: Kết quả huấn luyện lại mô hình RF sau Feature Engineering.....	54
Hình 35: Kết quả thử thêm feature cho mô hình RF	55
Hình 36: Kết quả thử nghiệm thêm với mô hình Gradient Boosting Regressor	56
Hình 37: Kết quả thử nghiệm điều chỉnh hyperparameter	57

1. GIỚI THIỆU

1.1 Giới thiệu bài toán

Trong thực tế, việc định giá bất động sản là một bài toán phức tạp vì giá trị của một căn nhà phụ thuộc vào hàng loạt yếu tố từ vị trí địa lý, diện tích xây dựng, năm xây dựng cho đến chất lượng nội thất và cơ sở hạ tầng xung quanh. Bài thực hành này hướng đến việc xây dựng một hệ thống dự đoán giá nhà tự động dựa trên kỹ thuật Machine Learning.

Dữ liệu được sử dụng là bộ dữ liệu House Prices – Advanced Regression Techniques, được công bố trên nền tảng Kaggle. Đây là tập dữ liệu thực tế bao gồm thông tin về các căn nhà tại Ames, Iowa (Mỹ) với hơn 79 đặc trưng mô tả nhiều khía cạnh của từng căn nhà như diện tích tổng thể, số lượng phòng ngủ, diện tích tầng hầm, chất lượng vật liệu hoàn thiện và nhiều thuộc tính khác. Biến mục tiêu cần dự đoán là SalePrice – mức giá bán thực tế của mỗi căn nhà.

Bài toán thuộc lớp bài toán hồi quy (regression) vì kết quả đầu ra là một giá trị số thực liên tục. Để giải quyết bài toán này, nhiều mô hình học máy được đề xuất và so sánh, bao gồm Linear Regression, Ridge, Lasso, Decision Tree và Random Forest Regressor.

1.2 Mục tiêu thực hiện

Bài thực hành đặt ra các mục tiêu cụ thể như sau:

- Nắm vững cơ chế hoạt động của bài toán hồi quy trong Machine Learning và sự khác biệt so với bài toán phân loại.
- Làm quen với quy trình chuẩn của một dự án học máy: từ khám phá dữ liệu, lựa chọn đặc trưng, huấn luyện mô hình đến đánh giá và cải tiến.
- Vận dụng các kỹ thuật phân tích tương quan để xác định những đặc trưng có giá trị cao đối với mô hình.
- Huấn luyện và so sánh hiệu năng của năm thuật toán hồi quy: Linear Regression, Ridge, Lasso, Decision Tree và Random Forest.
- Đánh giá mô hình qua ba chỉ số: MAE, RMSE và R^2 Score.

- Thực hiện Cross-Validation và Hyperparameter Tuning để đánh giá độ ổn định và tối ưu hóa mô hình.
- Thực hành Feature Engineering nhằm bổ sung thông tin hữu ích và cải thiện kết quả dự đoán.

2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về bài toán hồi quy

Bài toán hồi quy (Regression) là dạng bài toán học có giám sát (supervised learning) trong đó mô hình được đào tạo để dự đoán một biến đầu ra liên tục dựa trên tập hợp các biến đầu vào. Điểm khác biệt cốt lõi giữa hồi quy và phân loại nằm ở bản chất của nhãn: trong phân loại, nhãn là các lớp rời rạc (ví dụ: chó/mèo), còn trong hồi quy, nhãn là giá trị số thực (ví dụ: giá nhà, nhiệt độ, doanh thu).

Để đo độ sai lệch của mô hình so với thực tế, bài toán hồi quy sử dụng các hàm mất mát như Mean Squared Error (MSE) hay Mean Absolute Error (MAE). Mục tiêu học là tìm bộ tham số tối ưu giúp tối thiểu hóa những hàm mất mát này trên tập dữ liệu huấn luyện.

Hiểu một cách đơn giản về bài toán hồi quy, chúng ta sẽ huấn luyện cho mô hình học được những đặc trưng và mối quan hệ giữa dữ liệu đầu vào với giá trị đầu ra. Từ đó, mô hình có khả năng dự đoán các giá trị chưa từng xuất hiện trước đó với độ chính xác cao nhất có thể. Ví dụ, trong bài toán dự đoán giá nhà, mô hình sẽ học từ các thông tin như diện tích, số phòng, năm xây dựng, vị trí và chất lượng căn nhà để ước lượng giá bán phù hợp.

2.2 Linear Regression

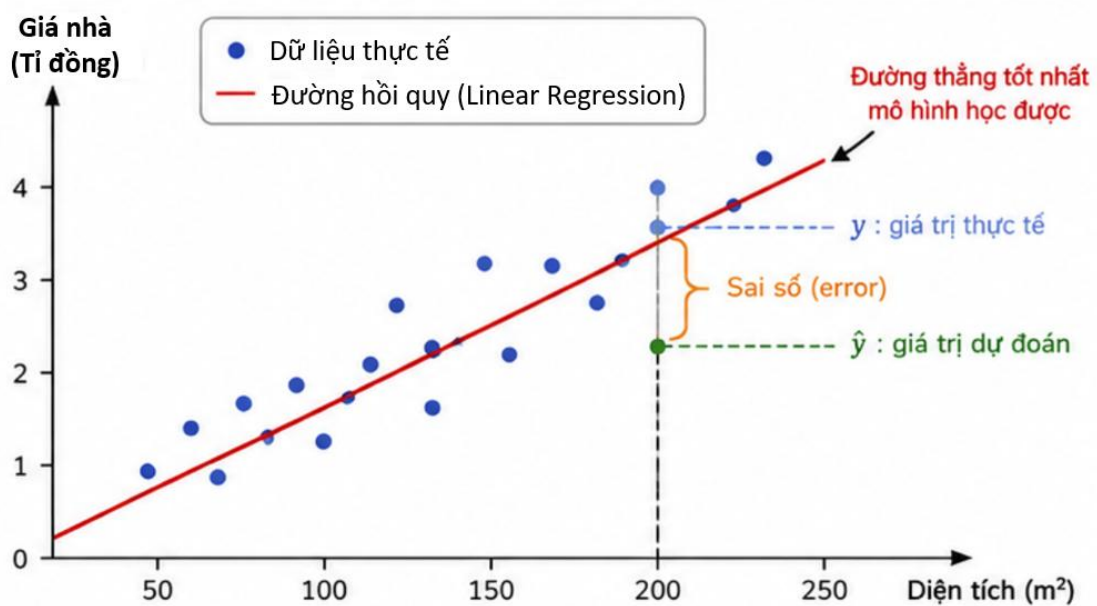
Linear Regression là thuật toán hồi quy cơ bản và phổ biến nhất trong Machine Learning. Thuật toán này giả định rằng tồn tại mối quan hệ tuyến tính giữa các đặc trưng đầu vào x và biến mục tiêu y . Mục tiêu của mô hình là tìm ra một đường thẳng hoặc siêu phẳng phù hợp nhất để biểu diễn mối quan hệ giữa dữ liệu đầu vào và đầu ra.

Phương trình tổng quát của mô hình được biểu diễn như sau:

$$\hat{y} = \omega_1 x_1 + \omega_2 x_2 + \dots + \omega_n x_n + b$$

Trong đó:

- $\hat{y}$: giá trị dự đoán.
- x : các feature đầu vào.
- ω : trọng số của từng feature.
- b : hệ số bias.



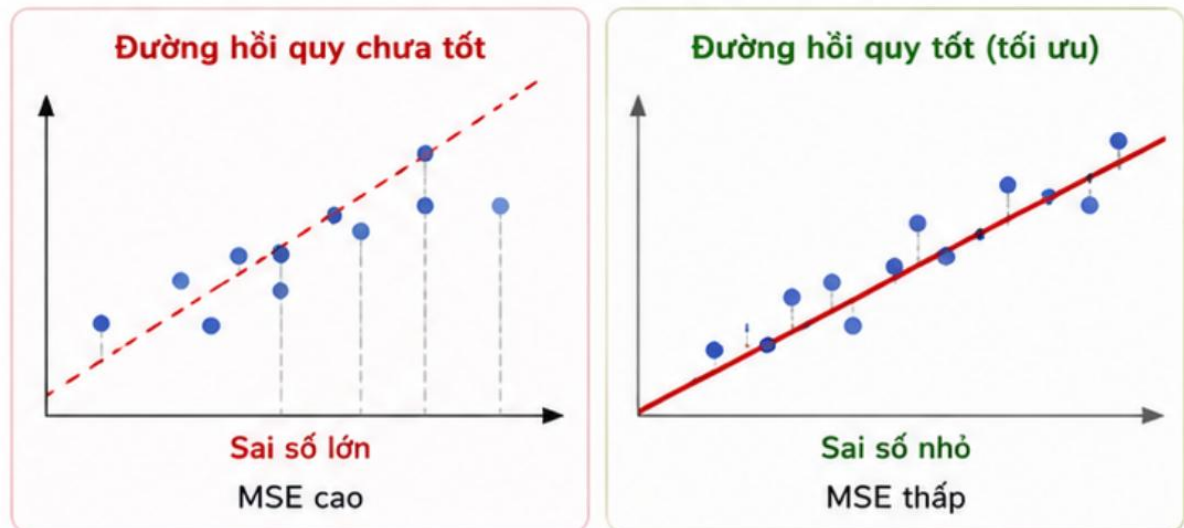
Hình 1: Minh họa thực tế bài toán hồi quy

Ví dụ minh họa đơn giản: trong bài toán dự đoán giá nhà, mô hình Linear Regression có thể học được rằng diện tích nhà càng lớn thì giá bán càng cao. Chẳng hạn, nếu diện tích tăng thêm 1 mét vuông thì giá nhà có thể tăng thêm một mức cố định tương ứng với trọng số của đặc trưng diện tích. Từ đó mô hình sử dụng các thông tin như diện tích, số phòng, năm xây dựng để đưa ra giá dự đoán cho những căn nhà mới chưa từng xuất hiện trong dữ liệu huấn luyện.

Trong quá trình huấn luyện, mô hình sẽ điều chỉnh các trọng số sao cho sai số giữa giá trị dự đoán và giá trị thực tế là nhỏ nhất. Các phương pháp tối ưu thường được sử dụng là Ordinary Least Squares (OLS) hoặc Gradient Descent. Hàm mất mát phổ biến nhất

của Linear Regression là Mean Squared Error (MSE), được tính bằng trung bình bình phương sai số giữa giá trị dự đoán và giá trị thực tế.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



Hình 2: Minh họa thực tế tiêu chuẩn đường hồi quy

Ưu điểm lớn nhất của Linear Regression là mô hình đơn giản, dễ triển khai và có khả năng diễn giải cao. Người dùng có thể dễ dàng quan sát mức độ ảnh hưởng của từng đặc trưng lên kết quả dự đoán thông qua các trọng số w_i . Ngoài ra, thời gian huấn luyện của mô hình cũng rất nhanh đối với các tập dữ liệu có kích thước vừa và nhỏ.

Tìm hiểu về các thực hiện tìm đường hồi quy bằng code

Bản chất của đoạn code là đi tìm đường thẳng có dạng:

Phương trình đường thẳng hồi quy có dạng:

$$y = mx + b$$

Trong đó:

- x là biến độc lập (Đầu vào: 30, 40, 50, 60).
- y là biến phụ thuộc (Đầu ra thực tế: 500, 600, 600, 750).
- m là hệ số góc (Quy định độ dốc của đường thẳng).
- b là hệ số chặn (Vị trí đường thẳng cắt trục tung Oy).

Thuật toán thực hiện việc này qua 4 bước lặp đi lặp lại như sau:

Bước 1: Khởi tạo giá trị ban đầu (Vẽ đường thẳng đầu tiên)

Lúc mới bắt đầu, máy tính chưa biết đường thẳng nào là tốt nhất. Code sẽ gán đại một giá trị: $m = -2$ và $b = 50$.

Khi vẽ lên đồ thị, đường thẳng này (màu cam dưới cùng) nằm lệch hoàn toàn so với các chấm dữ liệu thực tế (màu vàng). Sai số lúc này giữa đường thẳng dự đoán và thực tế là rất lớn.

Bước 2: Tính toán sai số và Độ dốc (Gradient)

Bên trong vòng lặp, máy tính sẽ tính toán xem đường thẳng hiện tại đang "tệ" đến mức nào bằng cách chạy qua từng điểm dữ liệu:

```
Python
y_pred = m * x[i] + b    # 1. Tính giá trị y dự đoán dựa trên đường thẳng hiện tại
error = y_pred - y[i]    # 2. Tính khoảng cách (sai số) giữa đường thẳng và điểm thực tế
```

Để biết phải sửa đường thẳng theo hướng nào (lên hay xuống, dốc hơn hay phẳng hơn), toán học sử dụng đạo hàm để tính ra **Độ dốc (Gradient)**:

- $db += 2 * error$: Đại diện cho việc cần dịch chuyển đường thẳng lên hay xuống.
- $dm += 2 * error * x[i]$: Đại diện cho việc cần xoay đường thẳng dốc lên hay dốc xuống.

Bước 3: Cập nhật tham số (Dịch chuyển đường thẳng)

Sau khi tính được hướng đi (dm và db), máy tính sẽ cập nhật lại giá trị của m và b bằng cách lấy giá trị cũ trừ đi một phần nhỏ của độ dốc (thông qua tốc độ học $n = 0.0001$):

```
Python
m = m - n * (dm / 4)
b = b - n * (db / 4)
```

Vì độ dốc ban đầu rất lớn, nên sau mỗi vòng lặp, đường thẳng sẽ được "kéo" lên một khoảng cách lớn (bạn có thể thấy các đường thẳng ban đầu cách xa nhau).

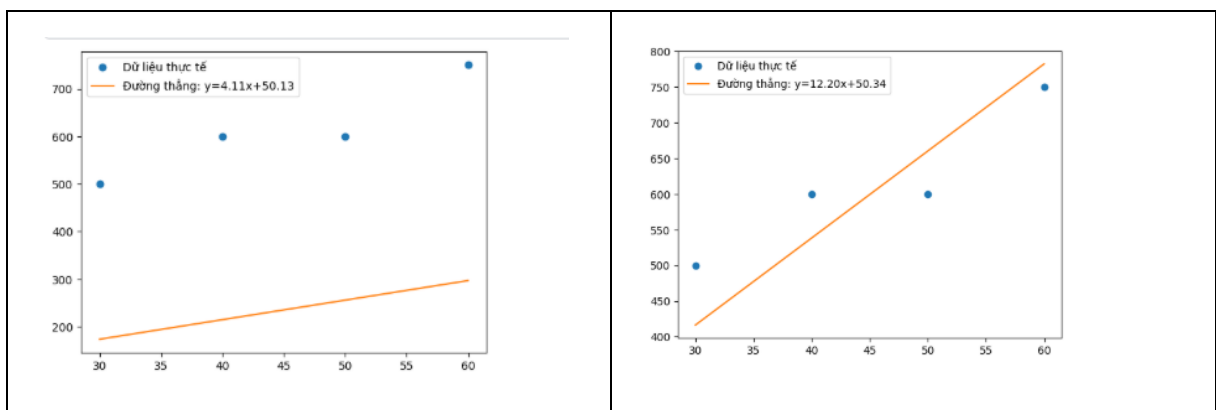
Bước 4: Hội tụ (Tìm ra đường thẳng tốt nhất)

Càng lặp lại nhiều lần (qua mỗi rep trong `range(100)`), đường thẳng càng tiến sát vào các chấm vàng.

Khi đường thẳng đã ở rất gần các điểm dữ liệu, sai số (error) giảm xuống gần bằng 0. Khi sai số bằng 0, độ dốc dm và db cũng bằng 0, khiến cho m và b dừng thay đổi. Hiện tượng các đường thẳng ở trên cùng xếp chồng khít lên nhau gọi là **Sự hội tụ**. Lúc này, thuật toán kết thúc và ta tìm được đường hồi quy tối ưu nhất cho dữ liệu (đường thẳng trên cùng).

```
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import clear_output
import time
x = np.array([30, 40, 50, 60])
y = np.array([500, 600, 600, 750])
m = -2
b = 50
n = 0.0001
N = len(x)
for rep in range(100):
    dm = 0
    db = 0
    for i in range(N):
        y_pred = m * x[i] + b
        error = y_pred - y[i]
        dm += 2 * error * x[i]
        db += 2 * error
    m = m - n * (dm / N)
    b = b - n * (db / N)
    clear_output(wait=True)
    plt.plot(x, y, 'o', label='Dữ liệu thực tế')
    plt.plot(x, m * x + b, label=f'Dường thẳng: y={m:.2f}x+{b:.2f}')
    plt.legend()
    #plt.show()
    time.sleep(0.1)
```

Kết quả

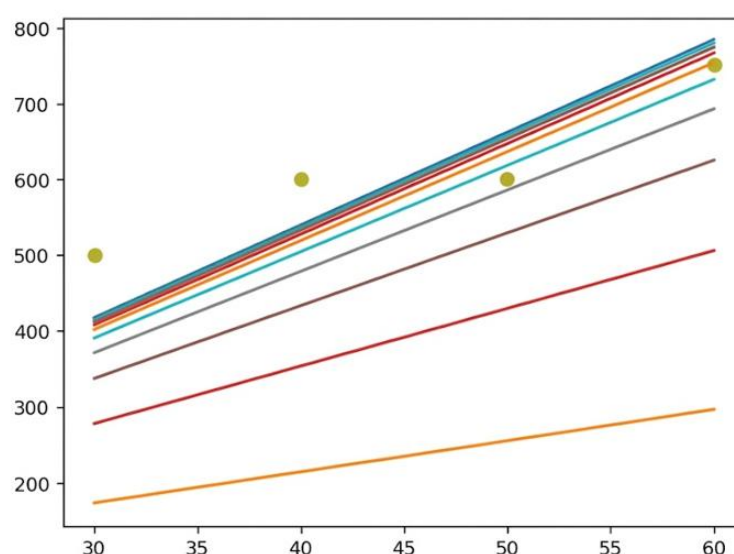


Hình 3: Kết quả dịch chuyển đường hồi quy

Nhận xét về quá trình dịch chuyển của đường hồi quy (Hình 3):

Giai đoạn khởi đầu (Hình 3 - Bên trái): Tại các vòng lặp đầu tiên, với hệ số ban đầu chưa tối ưu ($y = 4.11x + 50.13$), đường hồi quy nằm tách biệt và thấp hơn rất nhiều so với tập dữ liệu thực tế. Sai số tổng thể (Loss) ở giai đoạn này đạt giá trị lớn nhất.

Giai đoạn điều chỉnh (Hình 3 - Bên phải): Dưới tác động của thuật toán Gradient Descent, các hệ số liên tục được cập nhật. Đường hồi quy nhanh chóng được "kéo" lên phía trên với phương trình biến đổi thành $y = 12.20x + 50.34$. Độ dốc (hệ số góc m) tăng mạnh giúp đường thẳng bắt đầu xoay và bám sát tiến trình phân bố của các điểm dữ liệu thực tế.



Hình 4: Tất cả các đường hồi quy dịch chuyển đến khi hội tụ

Nhận xét về tính hội tụ của thuật toán (Hình 4)

Tốc độ thay đổi ở các pha: Đồ thị Hình 4 mô tả trực quan toàn bộ lịch sử tiến hóa của đường hồi quy qua các vòng lặp.

Ở các bước lặp đầu tiên (phía dưới), khoảng cách giữa các đường thẳng khá xa nhau, cho thấy thuật toán có tốc độ học nhanh, biên độ cập nhật tham số lớn nhờ giá trị độ dốc (Gradient) lớn.

Càng tiến về gần các điểm dữ liệu (phía trên), các đường thẳng càng xếp chồng khít và dày đặc lên nhau. Điều này chứng tỏ sai số đã giảm thiểu tối đa, độ dốc tiệm cận về mức 0, khiến các hệ số m và b chỉ thay đổi với biên độ rất nhỏ.

Kết luận chung: Thuật toán Gradient Descent đã hội tụ thành công. Đường thẳng trên cùng chính là đường hồi quy tuyến tính tối ưu nhất, phản ánh chính xác nhất xu hướng mối quan hệ giữa biến độc lập x và biến phụ thuộc y của tập dữ liệu mẫu.

2.3 Ridge Regression

Ridge Regression (L2 Regularization) là phiên bản cải tiến của Linear Regression, được thiết kế nhằm giảm hiện tượng overfitting và tăng khả năng tổng quát hóa của mô hình. Điểm khác biệt chính của Ridge Regression là bổ sung thêm một thành phần phạt (regularization term) vào hàm mất mát để kiểm soát độ lớn của các trọng số.

Hàm mất mát của Ridge Regression được biểu diễn như sau:

$$Loss = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m \omega_j^2$$

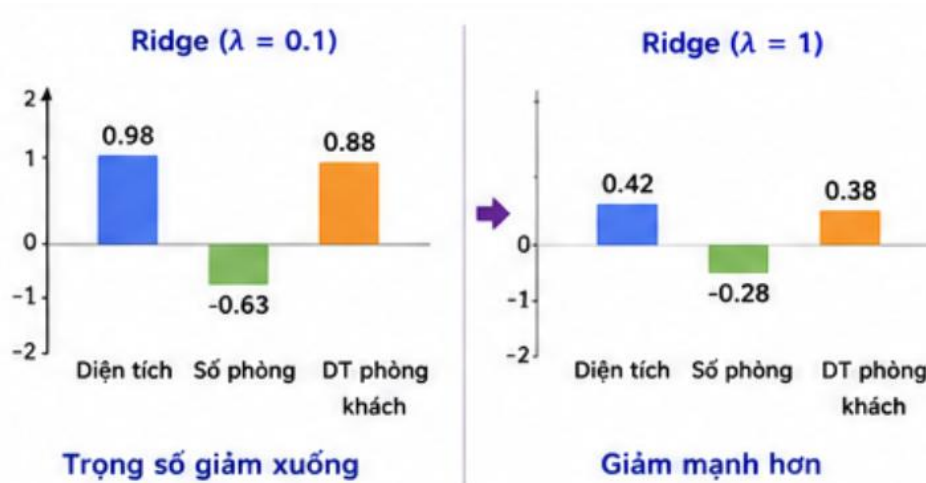
Trong đó:

- $\sum (y_i - \hat{y}_i)^2$ là tổng bình phương sai số giữa giá trị thực tế và giá trị dự đoán.
- λ là hệ số regularization.
- ω_j là các trọng số của mô hình.

Hệ số λ đóng vai trò điều chỉnh mức độ phạt đối với các trọng số lớn. Khi giá trị λ tăng, các trọng số sẽ bị ép nhỏ lại và tiến gần về 0, giúp mô hình giảm sự phụ thuộc quá mức vào một số đặc trưng cụ thể. Tuy nhiên, khác với Lasso Regression, Ridge không đưa trọng số về đúng bằng 0 mà chỉ làm giảm giá trị của chúng.

Một ưu điểm quan trọng của Ridge Regression là khả năng xử lý tốt hiện tượng đa cộng tuyến (multicollinearity), tức là khi các đặc trưng đầu vào có mối tương quan mạnh với nhau. Trong trường hợp này, Linear Regression thông thường có thể tạo ra các trọng số rất lớn và không ổn định, còn Ridge giúp mô hình ổn định hơn và cải thiện khả năng dự đoán.

Ví dụ minh họa: trong bài toán dự đoán giá nhà, các đặc trưng như diện tích nhà, số phòng và diện tích phòng khách thường có mối liên hệ với nhau. Nếu sử dụng Linear Regression thông thường, mô hình có thể bị phụ thuộc quá mức vào một vài đặc trưng và gây ra overfitting. Ridge Regression sẽ làm giảm độ lớn của các trọng số để mô hình học cân bằng hơn giữa các đặc trưng.



Hình 5: Minh họa thực tế ridge regression

Có thể hiểu đơn giản rằng Ridge Regression giống như việc “kiềm chế” mô hình không được học quá mạnh từ dữ liệu huấn luyện. Nhờ đó, mô hình có khả năng dự đoán tốt hơn trên dữ liệu mới và giảm nguy cơ sai lệch khi áp dụng thực tế.

2.4 Lasso Regression

Lasso Regression (L1 Regularization) là một biến thể của Linear Regression được bổ sung thêm thành phần regularization nhằm hạn chế hiện tượng overfitting và cải thiện khả năng tổng quát hóa của mô hình. Khác với Ridge Regression sử dụng bình phương trọng số, Lasso sử dụng giá trị tuyệt đối của trọng số để tạo thành phần phạt trong hàm mất mát.

Hàm mất mát của Lasso Regression được biểu diễn như sau:

$$Loss = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m |\omega_j|$$

Trong đó:

- $\sum (y_i - \hat{y}_i)^2$ là tổng bình phương sai số giữa giá trị thực tế và giá trị dự đoán.
- λ là hệ số regularization.
- w_j là các trọng số của mô hình.

Điểm đặc biệt quan trọng của Lasso Regression là khả năng đưa một số trọng số về đúng bằng 0 khi giá trị λ đủ lớn. Điều này đồng nghĩa với việc các đặc trưng tương ứng sẽ bị loại bỏ khỏi mô hình. Vì vậy, Lasso không chỉ giúp giảm overfitting mà còn thực hiện lựa chọn đặc trưng tự động (automatic feature selection).

Nhờ đặc tính này, mô hình Lasso thường tạo ra mô hình thưa (sparse model), tức là chỉ giữ lại các đặc trưng quan trọng nhất. Điều này giúp mô hình dễ diễn giải hơn và đặc biệt hữu ích trong các bài toán có số lượng đặc trưng lớn hoặc chứa nhiều đặc trưng nhiễu.

Ví dụ trong bài toán dự đoán giá nhà, giả sử tập dữ liệu có rất nhiều đặc trưng như diện tích, số phòng, màu sơn tường, loại cửa, kiểu mái nhà, vị trí gara,... Trong thực tế, không phải tất cả các đặc trưng đều ảnh hưởng mạnh đến giá bán. Lasso Regression sẽ tự động giảm trọng số của các đặc trưng ít quan trọng về 0 và chỉ giữ lại những đặc trưng thực sự có ảnh hưởng lớn như diện tích nhà, vị trí và chất lượng xây dựng.

Có thể hiểu đơn giản rằng Lasso Regression giống như quá trình “lọc bớt” các thông tin không cần thiết để mô hình tập trung học những đặc trưng quan trọng nhất. Nhờ đó, mô hình trở nên gọn hơn, dễ phân tích hơn và có khả năng hoạt động tốt hơn trên dữ liệu mới.

2.5 Decision Tree Regressor

Decision Tree Regressor là thuật toán hồi quy dựa trên cấu trúc cây quyết định (Decision Tree). Mô hình hoạt động bằng cách liên tục phân chia dữ liệu thành các nhóm nhỏ hơn dựa trên các điều kiện của đặc trưng đầu vào nhằm giảm thiểu sai số dự đoán. Đây là

mô hình phi tham số (non-parametric), nghĩa là không giả định trước mối quan hệ tuyến tính giữa dữ liệu đầu vào và đầu ra.

Trong quá trình huấn luyện, thuật toán sẽ tìm điểm chia tốt nhất tại mỗi node sao cho sai số bình phương trung bình (Mean Squared Error – MSE) của các nhóm sau khi phân chia là nhỏ nhất. Dữ liệu sẽ được chia thành các node con cho đến khi đạt điều kiện dừng như độ sâu tối đa hoặc số lượng mẫu tối thiểu trong node.

Tại mỗi node lá (leaf node), giá trị dự đoán được tính bằng trung bình của tất cả các mẫu thuộc node đó:

$$\hat{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

Trong đó:

- n là số lượng mẫu trong node lá.
- y_i là giá trị thực tế của từng mẫu.
- $\hat{y}$ là giá trị dự đoán của mô hình.

Ưu điểm lớn của Decision Tree Regressor là khả năng học được các mối quan hệ phi tuyến giữa các đặc trưng và biến mục tiêu. Mô hình cũng không yêu cầu chuẩn hóa dữ liệu và có thể xử lý tốt cả dữ liệu số lẫn dữ liệu dạng phân loại.

Ví dụ trong bài toán dự đoán giá nhà, cây quyết định có thể học theo dạng:

- Nếu diện tích nhà $> 150 \text{ m}^2$ và chất lượng xây dựng cao $\rightarrow$ giá nhà cao.
- Nếu diện tích nhỏ hơn $80 \text{ m}^2 \rightarrow$ giá nhà thấp.
- Nếu nhà mới xây và nằm ở vị trí tốt $\rightarrow$ giá tăng mạnh.

Nhờ cơ chế phân chia theo điều kiện, mô hình có thể mô tả được các quan hệ phức tạp mà Linear Regression khó biểu diễn.

Tuy nhiên, nếu cây được xây dựng quá sâu, mô hình sẽ ghi nhớ gần như toàn bộ dữ liệu huấn luyện dẫn đến hiện tượng overfitting. Khi đó mô hình hoạt động rất tốt trên tập

train nhưng dự đoán kém trên dữ liệu mới chưa từng xuất hiện. Vì vậy, các tham số như `max_depth`, `min_samples_split` hay `min_samples_leaf` thường được sử dụng để kiểm soát độ phức tạp của cây.

Có thể hiểu đơn giản rằng Decision Tree Regressor hoạt động giống như quá trình ra quyết định của con người: mô hình liên tục đặt câu hỏi và chia dữ liệu thành nhiều nhóm nhỏ hơn cho đến khi đưa ra được giá trị dự đoán phù hợp nhất.

2.6 Random Forest Regressor

Random Forest Regressor là thuật toán học ensemble (ensemble learning) được xây dựng bằng cách kết hợp nhiều cây quyết định (Decision Tree) nhằm nâng cao độ chính xác và khả năng tổng quát hóa của mô hình. Thay vì chỉ sử dụng một cây đơn lẻ, Random Forest tạo ra nhiều cây khác nhau và tổng hợp kết quả dự đoán từ tất cả các cây để đưa ra kết quả cuối cùng.

Mỗi cây trong Random Forest được huấn luyện trên một tập dữ liệu bootstrap, tức là tập dữ liệu được lấy mẫu ngẫu nhiên có lặp từ tập huấn luyện ban đầu. Ngoài ra, tại mỗi bước phân chia node, mô hình chỉ chọn ngẫu nhiên một phần các đặc trưng để tìm điểm chia tốt nhất. Cơ chế này giúp các cây trở nên đa dạng hơn và giảm sự phụ thuộc giữa các cây.

Giá trị dự đoán cuối cùng của Random Forest Regressor được tính bằng trung bình cộng kết quả dự đoán của tất cả các cây:

$$\hat{y} = \frac{1}{N} \sum_{i=1}^N T_i(x)$$

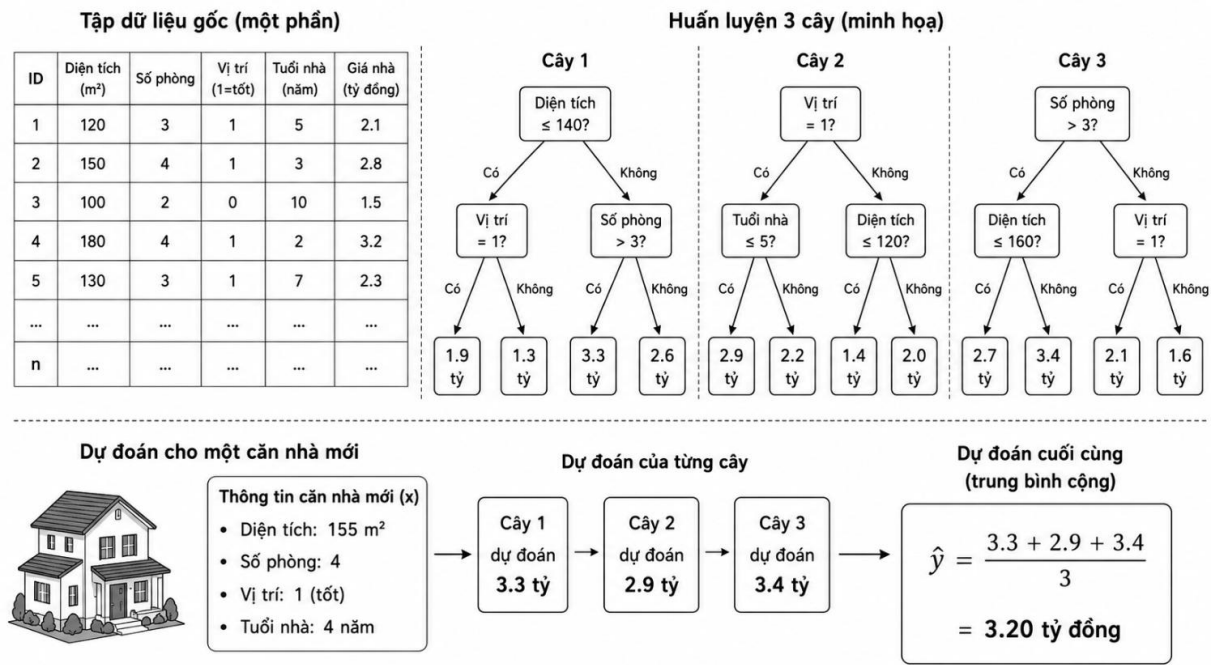
Trong đó:

- T là số lượng cây quyết định trong rừng.
- $T_i(x)$ là giá trị dự đoán của cây thứ i .
- $\hat{y}$ là kết quả dự đoán cuối cùng.

Ưu điểm nổi bật của Random Forest là khả năng giảm hiện tượng overfitting so với Decision Tree đơn lẻ. Trong khi một cây quyết định có thể học quá chi tiết dữ liệu huấn luyện, Random Forest sử dụng cơ chế bagging và ngẫu nhiên hóa đặc trưng để giảm phương sai (variance), từ đó giúp mô hình ổn định hơn trên dữ liệu mới.

Ví dụ trong bài toán dự đoán giá nhà:

- Cây thứ nhất có thể học mạnh về diện tích nhà.
- Cây thứ hai tập trung nhiều vào vị trí.
- Cây thứ ba quan tâm đến chất lượng xây dựng và năm xây dựng.



Hình 6: Minh họa thực tế random forest regression

Mỗi cây sẽ đưa ra một giá trị dự đoán khác nhau. Sau đó, Random Forest lấy trung bình tất cả các dự đoán để tạo ra kết quả cuối cùng ổn định và chính xác hơn.

Nhờ khả năng học được các mối quan hệ phi tuyến phức tạp, chống overfitting tốt và hoạt động hiệu quả trên nhiều loại dữ liệu khác nhau, Random Forest thường là một trong những mô hình được sử dụng phổ biến nhất trong các bài toán hồi quy thực tế. Trong nhiều trường hợp, đây cũng được xem là lựa chọn mặc định đáng tin cậy khi cần xây dựng mô hình dự đoán có độ chính xác cao.

2.7 Các chỉ số đánh giá mô hình

2.7.1 Mean Absolute Error (MAE)

Mean Absolute Error (MAE) là chỉ số dùng để đo lường độ sai lệch trung bình tuyệt đối giữa giá trị dự đoán của mô hình và giá trị thực tế. Đây là một trong những chỉ số đánh giá phổ biến nhất trong bài toán hồi quy do tính trực quan và dễ hiểu.

Công thức tính MAE được biểu diễn như sau:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Trong đó:

- y_i : giá trị thực tế
- $\hat{y}_i$: giá trị dự đoán
- n : số lượng mẫu dữ liệu

MAE cho biết trung bình mỗi dự đoán sai lệch bao nhiêu đơn vị so với giá trị thực tế. Giá trị MAE càng nhỏ thì mô hình dự đoán càng chính xác.

Ưu điểm của MAE là dễ diễn giải và ít bị ảnh hưởng bởi các giá trị ngoại lệ (outlier) vì sai số không bị bình phương như MSE hay RMSE. Tuy nhiên, MAE không phản ánh mạnh mức độ nghiêm trọng của các sai số lớn. Do đó, trong thực tế MAE thường được sử dụng kết hợp với RMSE và R^2 để đánh giá toàn diện hiệu năng của mô hình hồi quy.

2.7.2 Root Mean Squared Error (RMSE)

Root Mean Squared Error (RMSE) là chỉ số đánh giá mức độ sai lệch giữa giá trị dự đoán của mô hình và giá trị thực tế. RMSE được tính bằng căn bậc hai của trung bình bình phương sai số, do đó các sai số lớn sẽ bị phạt mạnh hơn so với MAE.

Công thức tính RMSE được biểu diễn như sau:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Trong đó:

- y_i : giá trị thực tế
- $\hat{y}_i$: giá trị dự đoán
- n : số lượng mẫu dữ liệu

Khác với MAE chỉ lấy giá trị tuyệt đối của sai số, RMSE sử dụng bình phương sai số nên các lỗi lớn sẽ được khuếch đại mạnh hơn. Điều này giúp RMSE phản ánh rõ hơn mức độ nghiêm trọng của các dự đoán sai lệch lớn.

2.7.3 R^2 Score (Coefficient of Determination)

R^2 Score (Coefficient of Determination) là chỉ số dùng để đánh giá mức độ mà mô hình có thể giải thích sự biến thiên của biến mục tiêu. Nói cách khác, R^2 cho biết mô hình học được bao nhiêu phần trăm thông tin từ dữ liệu thực tế.

Công thức tính R^2 được biểu diễn như sau:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Trong đó:

- y_i : giá trị thực tế
- $\hat{y}_i$: giá trị dự đoán
- $\bar{y}$: giá trị trung bình của dữ liệu thực tế

Giá trị R^2 nằm trong khoảng từ 0 đến 1:

- R^2 càng gần 1 thì mô hình càng tốt
- R^2 gần 0 cho thấy mô hình dự đoán chưa hiệu quả

3. QUY TRÌNH THỰC HIỆN

3.1 Chuẩn bị và tải dữ liệu

Bộ dữ liệu *House Prices – Advanced Regression Techniques* được tải từ Kaggle và bao gồm hai tập dữ liệu chính:

- `train.csv`: gồm 1460 mẫu dữ liệu với 81 thuộc tính.
- `test.csv`: gồm 1459 mẫu dữ liệu với 80 thuộc tính.

Trong đó, tập `train.csv` chứa biến mục tiêu `SalePrice` dùng làm nhãn cho quá trình huấn luyện mô hình, trong khi tập `test.csv` không chứa nhãn và được sử dụng để đánh giá kết quả dự đoán trên hệ thống Kaggle.

Bộ dữ liệu bao gồm nhiều đặc trưng liên quan đến nhà ở như diện tích, năm xây dựng, số phòng, chất lượng vật liệu, vị trí và các thông tin kiến trúc khác. Đây là bài toán hồi quy với mục tiêu dự đoán giá bán của căn nhà dựa trên các đặc trưng đầu vào.

3.2 Khám phá và phân tích dữ liệu

Sau khi tải dữ liệu, bước đầu tiên là tiến hành khám phá dữ liệu (Exploratory Data Analysis – EDA) nhằm hiểu rõ đặc điểm của tập dữ liệu trước khi xây dựng mô hình.

Các bước phân tích bao gồm:

- Kiểm tra kích thước tập dữ liệu.
- Kiểm tra kiểu dữ liệu của từng thuộc tính.
- Thống kê mô tả cơ bản như giá trị nhỏ nhất, lớn nhất, trung bình và độ lệch chuẩn.
- Phân tích tỷ lệ dữ liệu thiếu của từng cột.

Tiếp theo, tiến hành khảo sát phân phối của biến mục tiêu `SalePrice` thông qua biểu đồ histogram nhằm đánh giá mức độ phân bố dữ liệu. Đồng thời, mối quan hệ giữa các

đặc trưng số và `SalePrice` được phân tích bằng scatter plot và correlation matrix để xác định các đặc trưng có ảnh hưởng mạnh đến giá nhà.

Thông qua bước này, có thể xác định được những đặc trưng quan trọng cũng như phát hiện các dữ liệu ngoại lệ (outliers) ảnh hưởng đến quá trình huấn luyện mô hình.

3.3 Xử lý dữ liệu thiếu và lựa chọn đặc trưng

Trong dữ liệu thực tế, việc xuất hiện các giá trị thiếu (missing values) là điều phổ biến. Một số thuộc tính trong bộ dữ liệu có tỷ lệ thiếu cao do căn nhà không sở hữu đặc điểm tương ứng, ví dụ như `PoolQC` bị thiếu khi căn nhà không có hồ bơi.

Sau khi phân tích tỷ lệ dữ liệu thiếu, các đặc trưng số có mức tương quan cao với `SalePrice` được lựa chọn để đưa vào mô hình học máy. Việc lựa chọn đặc trưng giúp giảm nhiễu dữ liệu và cải thiện hiệu năng dự đoán.

Đối với các giá trị thiếu trong các cột số, phương pháp thay thế bằng giá trị trung bình (mean imputation) được áp dụng nhằm đảm bảo dữ liệu đầu vào đầy đủ trước khi huấn luyện mô hình.

3.4 Phân chia tập dữ liệu và huấn luyện mô hình

Tập dữ liệu `train.csv` được chia thành hai phần:

- 80% dữ liệu dùng cho huấn luyện (training set).
- 20% dữ liệu dùng cho kiểm định (validation set).

Quá trình chia dữ liệu sử dụng `random_state= 42` nhằm đảm bảo khả năng tái hiện kết quả trong các lần chạy khác nhau.

Sau đó, các mô hình hồi quy bao gồm:

- Linear Regression
- Ridge Regression
- Lasso Regression

- Decision Tree Regressor
- Random Forest Regressor

được tiến hành huấn luyện và đánh giá trên tập validation thông qua các chỉ số:

- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)
- R^2 Score

Ngoài ra, kỹ thuật Cross-Validation 5-fold cũng được áp dụng đối với mô hình Random Forest nhằm đánh giá tính ổn định và khả năng tổng quát hóa của mô hình trên nhiều tập dữ liệu khác nhau.

3.5 Tối ưu hóa và cải thiện mô hình

Sau bước đánh giá ban đầu, các kỹ thuật cải thiện mô hình được áp dụng nhằm nâng cao độ chính xác dự đoán. Đầu tiên, phương pháp Hyperparameter Tuning sử dụng GridSearchCV được triển khai để tìm ra bộ siêu tham số tối ưu cho mô hình Random Forest.

Tiếp theo, kỹ thuật Feature Engineering được áp dụng để xây dựng thêm các đặc trưng mới mang ý nghĩa thực tiễn, bao gồm:

Cuối cùng, mô hình Gradient Boosting Regressor cũng được thử nghiệm nhằm so sánh hiệu năng với Random Forest và đánh giá khả năng cải thiện kết quả dự đoán.

3.6 Môi trường và công cụ phát triển

Toàn bộ quá trình thực nghiệm được thực hiện bằng ngôn ngữ Python trên nền tảng Kaggle Notebook với cấu hình GPU T4 x2.

Các thư viện chính được sử dụng bao gồm:

- **NumPy, Pandas:** xử lý và phân tích dữ liệu.

- **Matplotlib, Seaborn:** trực quan hóa dữ liệu.
- **Scikit-learn (sklearn):** xây dựng và huấn luyện các mô hình Machine Learning.

Ngoài ra, môi trường Kaggle Notebook hỗ trợ thuận tiện cho việc chạy thực nghiệm, lưu trữ dữ liệu và đánh giá kết quả trực tiếp trên nền tảng trực tuyến.

4. KẾT QUẢ THỰC NGHIỆM

Phần này trình bày toàn bộ kết quả thực nghiệm theo từng bài tập của bài thực hành, bao gồm code thực thi, kết quả đạt được và nhận xét phân tích chi tiết cho từng bước trong quy trình Machine Learning.

4.1 Các Bước Khởi Tạo Ban Đầu

4.1.1 Chuẩn bị môi trường

Cài đặt và import các thư viện cần thiết cho quá trình xử lý dữ liệu, xây dựng và đánh giá mô hình Machine Learning:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
```

4.1.2 Tải dữ liệu

```
train = pd.read_csv('/kaggle/input/competitions/house-prices-advanced-regression-techniques/train.csv')

test = pd.read_csv('/kaggle/input/competitions/house-prices-advanced-regression-techniques/test.csv')

print(train.shape)

train.head()
```

(1460, 81)

[5]:

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType	SaleCondition	SalePrice
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2008	WD	Normal	208500
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	2007	WD	Normal	181500
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9	2008	WD	Normal	223500
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2006	WD	Abnorml	140000
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	12	2008	WD	Normal	250000

5 rows x 81 columns

Hình 7: Tổng quan về dữ liệu trên kaggle

Sau khi tải bộ dữ liệu train.csv, tiến hành kiểm tra tổng quan dữ liệu để xác định cấu trúc và đặc điểm ban đầu của tập dữ liệu. Kết quả cho thấy dữ liệu có kích thước: (1460,81)

Điều này cho thấy bộ dữ liệu gồm 1460 mẫu và 81 thuộc tính, trong đó SalePrice là biến mục tiêu cần dự đoán. Dữ liệu bao gồm cả thuộc tính số và thuộc tính phân loại mô tả các đặc điểm của căn nhà như diện tích, số phòng, năm xây dựng và chất lượng công trình. Ngoài ra, một số cột xuất hiện giá trị thiếu (NaN), do đó cần thực hiện bước tiền xử lý dữ liệu trước khi huấn luyện mô hình.

4.1.3 Kiểm tra thông tin tổng quan của dataset

```
train.info()

train.describe()
```

Kích thước dataset: (1460, 81) <class 'pandas.core.frame.DataFrame'> RangeIndex: 1460 entries, 0 to 1459 Data columns (total 81 columns): # Column Non-Null Count Dtype --- --- --- --- 0 Id 1460 non-null int64 1 MSSubClass 1460 non-null int64 2 MSZoning 1460 non-null object 3 LotFrontage 1201 non-null float64 4 LotArea 1460 non-null int64 5 Street 1460 non-null object 6 Alley 91 non-null object 7 LotShape 1460 non-null object 8 LandContour 1460 non-null object 9 Utilities 1460 non-null object 10 LotConfig 1460 non-null object 11 LandSlope 1460 non-null object 12 Neighborhood 1460 non-null object 13 Condition1 1460 non-null object 14 Condition2 1460 non-null object 15 BldgType 1460 non-null object 16 HouseStyle 1460 non-null object 17 OverallQual 1460 non-null int64 18 OverallCond 1460 non-null int64 19 YearBuilt 1460 non-null int64 20 YearRemodAdd 1460 non-null int64 21 RoofStyle 1460 non-null object 22 RoofMatl 1460 non-null object 23 Exterior1st 1460 non-null object 24 Exterior2nd 1460 non-null object 25 MasVnrType 588 non-null object 26 MasVnrArea 1452 non-null float64 27 ExterQual 1460 non-null object 28 ExterCond 1460 non-null object 29 Foundation 1460 non-null object 30 BsmtQual 1423 non-null object 31 BsmtCond 1423 non-null object 32 BsmtExposure 1422 non-null object 33 BsmtFinType1 1423 non-null object 34 BsmtFinSF1 1460 non-null int64 35 BsmtFinType2 1422 non-null object 36 BsmtFinSF2 1460 non-null int64 37 BsmtUnfSF 1460 non-null int64 38 TotalBsmtSF 1460 non-null int64 39 Heating 1460 non-null object				40 HeatingQC 1460 non-null object 41 CentralAir 1460 non-null object 42 Electrical 1459 non-null object 43 1stFlrSF 1460 non-null int64 44 2ndFlrSF 1460 non-null int64 45 LowQualFinSF 1460 non-null int64 46 GrLivArea 1460 non-null int64 47 BsmtFullBath 1460 non-null int64 48 BsmtHalfBath 1460 non-null int64 49 FullBath 1460 non-null int64 50 HalfBath 1460 non-null int64 51 BedroomAbvGr 1460 non-null int64 52 KitchenAbvGr 1460 non-null int64 53 KitchenQual 1460 non-null object 54 TotRmsAbvGrd 1460 non-null int64 55 Functional 1460 non-null object 56 Fireplaces 1460 non-null int64 57 FireplaceQu 770 non-null object 58 GarageType 1379 non-null object 59 GarageYrBlt 1379 non-null float64 60 GarageFinish 1379 non-null object 61 GarageCars 1460 non-null int64 62 GarageArea 1460 non-null int64 63 GarageQual 1379 non-null object 64 GarageCond 1379 non-null object 65 PavedDrive 1460 non-null object 66 WoodDeckSF 1460 non-null int64 67 OpenPorchSF 1460 non-null int64 68 EnclosedPorch 1460 non-null int64 69 3SsnPorch 1460 non-null int64 70 ScreenPorch 1460 non-null int64 71 PoolArea 1460 non-null int64 72 PoolQC 7 non-null object 73 Fence 281 non-null object 74 MiscFeature 54 non-null object 75 MiscVal 1460 non-null int64 76 MoSold 1460 non-null int64 77 YrSold 1460 non-null int64 78 SaleType 1460 non-null object 79 SaleCondition 1460 non-null object 80 SalePrice 1460 non-null int64 dtypes: float64(3), int64(35), object(43) memory usage: 924.0+ KB
---	--	--	--	--

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	...	WoodDeckSF	OpenPorchSF	EnclosedPorch
count	1460.000000	1460.000000	1201.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1452.000000	1460.000000	...	1460.000000	1460.000000	1460.000000
mean	730.500000	56.897260	70.049958	10516.828082	6.099315	5.575342	1971.267808	1984.865753	103.685262	443.639726	...	94.244521	46.660274	21.954111
std	421.610009	42.300571	24.284752	9981.264932	1.382997	1.112799	30.202904	20.645407	181.066207	456.098091	...	125.338794	66.256028	61.119144
min	1.000000	20.000000	21.000000	1300.000000	1.000000	1.000000	1872.000000	1950.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000
25%	365.750000	20.000000	59.000000	7553.500000	5.000000	5.000000	1954.000000	1967.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000
50%	730.500000	50.000000	69.000000	9478.500000	6.000000	5.000000	1973.000000	1994.000000	0.000000	383.500000	...	0.000000	25.000000	0.000000
75%	1095.250000	70.000000	80.000000	11601.500000	7.000000	6.000000	2000.000000	2004.000000	166.000000	712.250000	...	168.000000	68.000000	0.000000
max	1460.000000	190.000000	313.000000	215245.000000	10.000000	9.000000	2010.000000	2010.000000	1600.000000	5644.000000	...	857.000000	547.000000	552.000000
8 rows × 38 columns														

WoodDeckSF	OpenPorchSF	EnclosedPorch	3SsnPorch	ScreenPorch	PoolArea	MiscVal	MoSold	YrSold	SalePrice
1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000
94.244521	46.660274	21.954110	3.409589	15.060959	2.758904	43.489041	6.321918	2007.815753	180921.195890
125.338794	66.256028	61.119149	29.317331	55.757415	40.177307	496.123024	2.703626	1.328095	79442.502883
0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	2006.000000	34900.000000
0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	5.000000	2007.000000	129975.000000
0.000000	25.000000	0.000000	0.000000	0.000000	0.000000	0.000000	6.000000	2008.000000	163000.000000
168.000000	68.000000	0.000000	0.000000	0.000000	0.000000	0.000000	8.000000	2009.000000	214000.000000
857.000000	547.000000	552.000000	508.000000	480.000000	738.000000	15500.000000	12.000000	2010.000000	755000.000000

Hình 8: Thông tin về dataset

Sau khi kiểm tra thông tin tổng quan của bộ dữ liệu bằng hàm `info()` và `describe()`, có thể nhận thấy tập dữ liệu gồm 1460 mẫu và 81 thuộc tính với hai kiểu dữ liệu chính là `int64`, `float64` và `object`. Trong đó, các thuộc tính dạng số được sử dụng để mô tả diện tích, số phòng, năm xây dựng và nhiều đặc điểm định lượng khác của căn nhà.

Kết quả thống kê mô tả cho thấy giá trị của các thuộc tính có độ phân tán khá lớn. Ví dụ, biến mục tiêu `SalePrice` có giá trị trung bình khoảng 180796 USD, giá nhỏ nhất khoảng 34900 USD và giá lớn nhất lên đến 755000 USD. Điều này cho thấy dữ liệu có sự biến động mạnh về giá nhà.

Ngoài ra, một số thuộc tính xuất hiện số lượng lớn giá trị thiếu như `PoolQC`, `MiscFeature`, `Alley` và `Fence`. Đây là đặc điểm phổ biến trong dữ liệu thực tế và cần được xử lý trước khi huấn luyện mô hình nhằm tránh ảnh hưởng đến hiệu năng dự đoán.

4.2 Data Exploration (Bài tập 1)

Thực hiện bước khám phá dữ liệu (Exploratory Data Analysis – EDA) nhằm hiểu rõ quy mô và cấu trúc của bộ dữ liệu trước khi tiến hành xây dựng mô hình học máy. Đầu tiên, kiểm tra kích thước của tập `train` và `test` để xác định số lượng mẫu dữ liệu và số lượng thuộc tính bằng lệnh:

4.2.1 Kích thước của `train` và `test dataset`

Kiểm tra kích thước dataset nhằm xác định số lượng mẫu và số lượng đặc trưng:

```
print("Train shape:", train.shape)
```

```
print("Test shape:", test.shape)
```

Train shape: (1460, 81)
Test shape: (1459, 80)

Hình 9: Kích thước về dataset

Kết quả cho thấy tập `train` gồm 1460 mẫu với 81 thuộc tính, trong khi tập `test` gồm 1459 mẫu với 80 thuộc tính. Sự khác biệt này xuất phát từ việc tập `train` chứa thêm cột `SalePrice`, đây là biến mục tiêu dùng để huấn luyện mô hình hồi quy dự đoán giá nhà.

4.2.2 Kiểm tra các cột có missing values

Phát hiện các cột chứa dữ liệu thiếu để lựa chọn phương pháp xử lý phù hợp:

```
missing = train.isnull().sum()  
missing = missing[missing > 0]  
print(missing.sort_values(ascending=False))
```

LotFrontage	259
Alley	1369
MasVnrType	872
MasVnrArea	8
BsmtQual	37
BsmtCond	37
BsmtExposure	38
BsmtFinType1	37
BsmtFinType2	38
Electrical	1
FireplaceQu	690
GarageType	81
GarageYrBlt	81
GarageFinish	81
GarageQual	81
GarageCond	81
PoolQC	1453
Fence	1179
MiscFeature	1406
dtype:	int64

Hình 10: Tổng hợp dữ liệu trong tập bị thiếu

Nhận xét: Kết quả khảo sát cho thấy tập dữ liệu tồn tại tỷ lệ giá trị thiếu (*missing values*) khá cao ở một số thuộc tính cụ thể. Nguyên nhân chủ yếu xuất phát từ đặc tính thực tế của bất động sản: không phải ngôi nhà nào cũng được trang bị hồ bơi (PoolQC), hàng rào (Fence) hay lò sưởi (FireplaceQu). Do đó, các trường thông tin này bị bỏ trống một cách logic (mang ý nghĩa "Không có") chứ không hẳn là lỗi thu thập. Bước kiểm tra này là cơ sở quan trọng để đưa ra chiến lược xử lý dữ liệu khuyết thiếu phù hợp (như thay thế bằng giá trị "None" hoặc số 0) trước khi đưa vào mô hình huấn luyện.

4.2.3 Tính tỷ lệ phần trăm missing values của từng cột

Tỷ lệ phần trăm giá trị thiếu (Missing Values) xác định mức độ khuyết dữ liệu của từng thuộc tính trong tập dữ liệu. Thông qua tỷ lệ này, chúng ta có thể lựa chọn phương pháp xử lý (imputation hoặc loại bỏ) phù hợp nhằm đảm bảo chất lượng dữ liệu đầu vào trước khi huấn luyện mô hình.

$$Missing\% = \frac{\text{Số ô thiếu}}{\text{Tổng số dòng}} \times 100$$

```
missing_percent = train.isnull().mean() * 100
missing_percent = missing_percent[missing_percent > 0]
print(missing_percent.sort_values(ascending=False))
```

PoolQC	99.520548
MiscFeature	96.301370
Alley	93.767123
Fence	80.753425
MasVnrType	59.726027
FireplaceQu	47.260274
LotFrontage	17.739726
GarageType	5.547945
GarageYrBlt	5.547945
GarageFinish	5.547945
GarageQual	5.547945
GarageCond	5.547945
BsmtExposure	2.602740
BsmtFinType2	2.602740
BsmtQual	2.534247
BsmtCond	2.534247
BsmtFinType1	2.534247
MasVnrArea	0.547945
Electrical	0.068493
dtype:	float64

Hình 11: Tỷ lệ missing values của từng cột

Kết quả: thống kê cho thấy sự phân hóa rõ rệt về tỷ lệ khuyết dữ liệu giữa các đặc trưng. Một số thuộc tính như **PoolQC**, **MiscFeature**, **Alley**, và **Fence** có tỷ lệ thiếu cực kỳ cao (trên 80%), do đó cần xem xét loại bỏ hoặc xử lý đặc biệt bằng cách gán nhãn "Không có" (None) để tránh làm nhiễu mô hình. Ngược lại, đối với các cột có tỷ lệ thiếu rất thấp (dưới 5%) như **MasVnrType**, **MasVnrArea**, hay **Electrical**, phương pháp điền khuyết bằng giá trị trung vị (*median*) hoặc giá trị xuất hiện nhiều nhất (*mode*) sẽ được áp dụng nhằm duy trì tính toàn vẹn và tối ưu hóa chất lượng dữ liệu đầu vào cho các thuật toán Machine Learning.

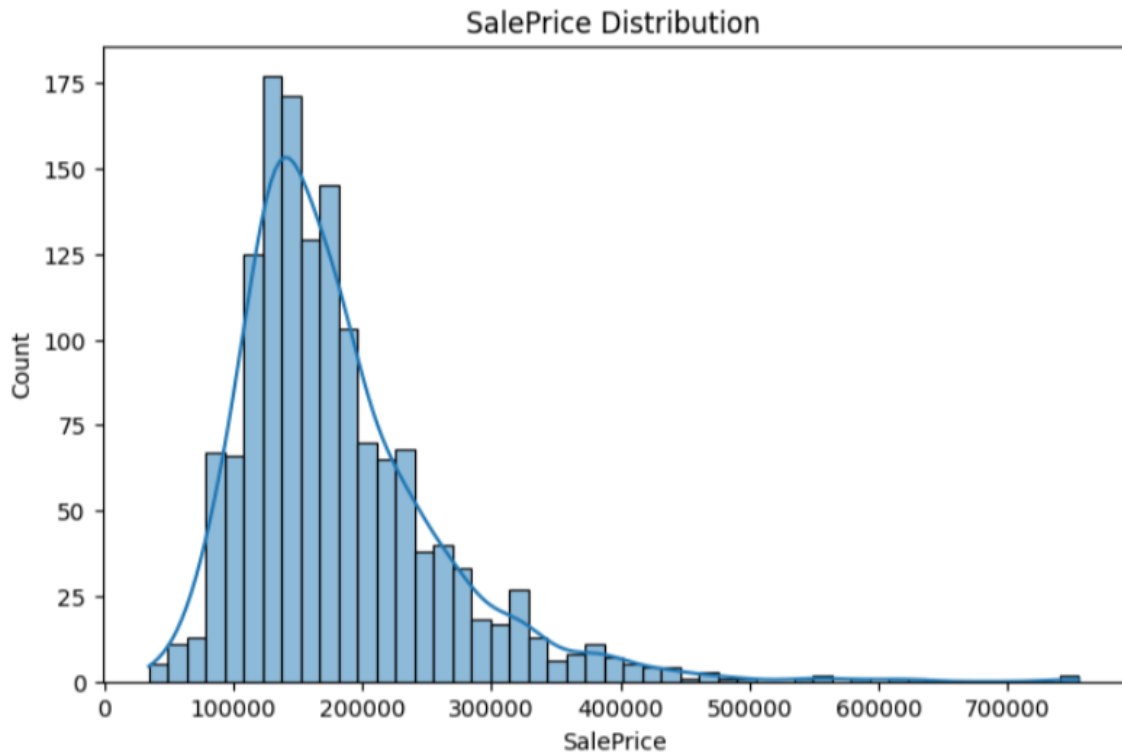
4.3 Visualization (Bài tập 2)

Trong bài thực hành này, bước trực quan hóa dữ liệu được thực hiện nhằm quan sát hình thái phân bố của giá nhà và mối quan hệ giữa các đặc trưng (features) với biến mục tiêu **SalePrice**. Việc trực quan hóa dữ liệu giúp dễ dàng phát hiện xu hướng, đánh giá mối tương quan giữa các biến, đồng thời nhận diện sớm các điểm ngoại lệ (outliers) trong tập dữ liệu. Các kỹ thuật đồ họa được sử dụng bao gồm biểu đồ tần suất (Histogram) và biểu đồ phân tán (Scatter Plot) để phân tích sâu mức độ ảnh hưởng của diện tích không gian sống (**GrLivArea**) cùng chất lượng xây dựng tổng thể (**OverallQual**) đến giá trị bất động sản.

4.3.1 Histogram của SalePrice

Trong phần này, biến mục tiêu **SalePrice** được trực quan hóa bằng biểu đồ Histogram kết hợp đường ước lượng mật độ nhân (KDE) nhằm phân tích chi tiết quy luật phân bố của giá nhà trong dataset.

```
plt.figure(figsize=(8,5))
sns.histplot(train['SalePrice'], kde=True)
plt.title("SalePrice Distribution")
plt.show()
```



Hình 12: Biểu đồ về sự phân bố giá nhà

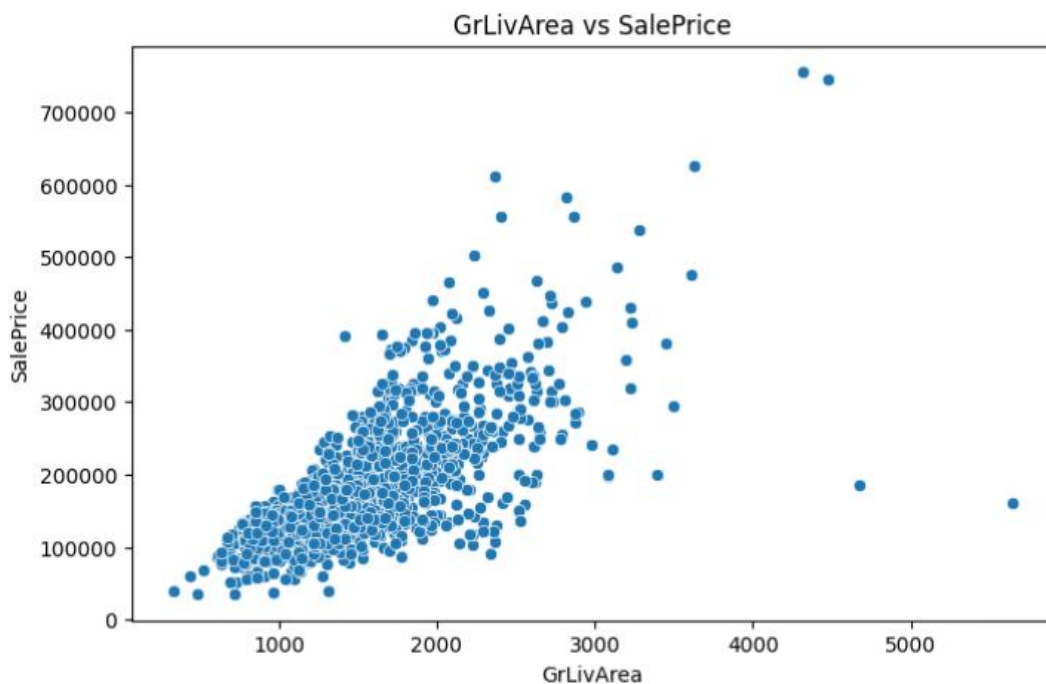
Nhận xét:

Biểu đồ trực quan hóa cho thấy phần lớn các căn nhà trong tập dữ liệu có giá bán tập trung phổ biến trong phân khúc từ \$100,000\$ đến \$250,000\$ USD. Đây là khoảng giá có mật độ xuất hiện cao nhất, đại diện cho phân khúc bất động sản trung cấp. Khi giá trị căn nhà tăng dần, tần suất xuất hiện của các mẫu dữ liệu giảm đi rõ rệt. Tập dữ liệu ghi nhận rất ít các căn nhà thuộc phân khúc cao cấp có giá từ \$400,000\$ USD trở lên. Đặc điểm này khiến đường cong phân bố kéo một "cái đuôi" dài về phía bên phải, minh chứng rằng biến SalePrice sở hữu thuộc tính phân bố lệch phải (Right-skewed / Positively skewed). Hiện tượng này gợi ý rằng việc áp dụng phép biến đổi logarit cho biến mục tiêu ở các bước tiền xử lý nâng cao sẽ là giải pháp cần thiết để chuẩn hóa dữ liệu trước khi đưa vào mô hình hồi quy.

4.3.2 Scatter plot giữa GrLivArea và SalePrice

Trong phần này, mối quan hệ giữa diện tích không gian sống trên mặt đất (**GrLivArea**) và giá bán nhà (**SalePrice**) được trực quan hóa bằng biểu đồ phân tán nhằm đánh giá xu hướng tương quan và phát hiện các điểm ngoại lệ cấu trúc.

```
plt.figure(figsize=(8,5))
sns.scatterplot(
    x=train['GrLivArea'],
    y=train['SalePrice']
)
plt.title("GrLivArea vs SalePrice")
plt.show()
```



Hình 13: Biểu đồ mối quan hệ giữa diện tích sử dụng của căn nhà và giá bán

Nhận xét:

Biểu đồ phân tán cho thấy một mối tương quan thuận tuyến tính rất rõ rệt giữa biến GrLivArea và SalePrice. Khi diện tích không gian sống trên mặt đất tăng lên, giá trị của căn nhà cũng có xu hướng tăng theo tỷ lệ thuận. Phần lớn các mẫu dữ liệu tập trung đậm

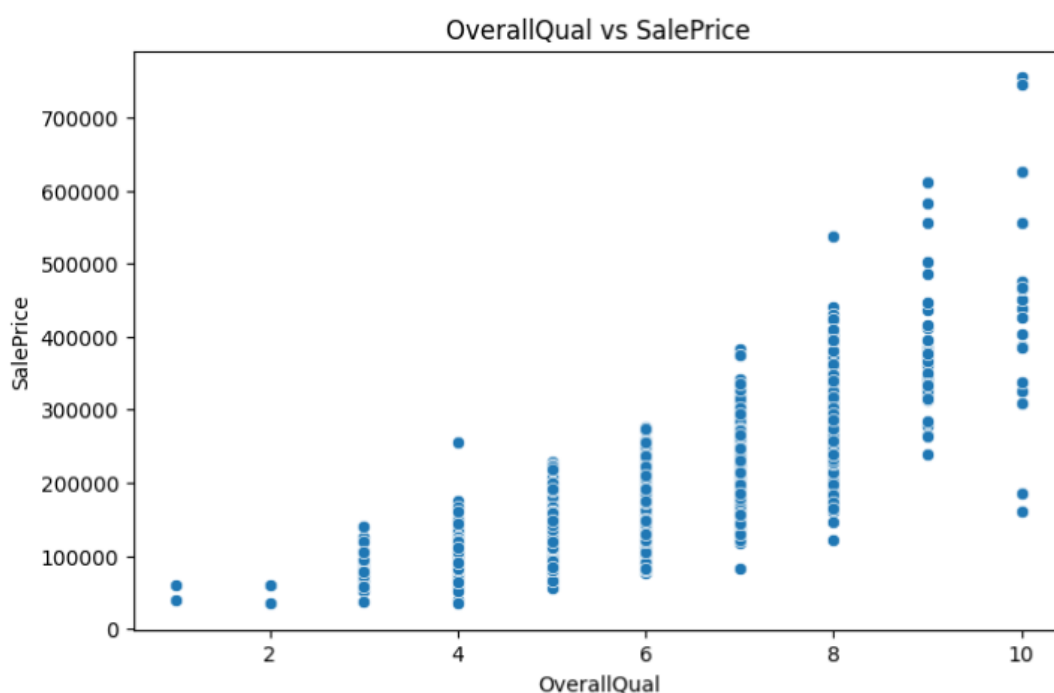
đặc trong khoảng diện tích từ 1.000 đến 2.500 sqft, tương ứng với mức giá từ 100.000 đến 300.000 USD.

Đáng chú ý, biểu đồ xuất hiện một số điểm dị biệt (*outliers*) rất điển hình ở góc phía dưới bên phải: có hai căn nhà có diện tích cực kỳ lớn (trên 4.500 sqft và gần 5.600 sqft) nhưng giá bán lại thấp bất thường (dưới 200.000 USD). Đây là những điểm nhiễu nghiêm trọng trong thực tế (có thể do nhà ở vùng cô lập hoặc chưa hoàn thiện) và cần được xem xét loại bỏ trong bước tiền xử lý để tránh làm sai lệch đường tiệm cận của các mô hình hồi quy. Nhìn chung, diện tích sử dụng là một đặc trưng mang tính quyết định và có giá trị dự báo cực kỳ cao đối với biến mục tiêu.

4.3.3 Scatter plot giữa OverallQual và SalePrice

Trong phần này, mối quan hệ giữa chất lượng vật liệu và mức độ hoàn thiện tổng thể của căn nhà (**OverallQual**) với giá bán (**SalePrice**) được trực quan hóa bằng biểu đồ phân tán nhằm đánh giá mức độ ảnh hưởng của một đặc trưng thứ bậc đến biến mục tiêu.

```
plt.figure(figsize=(8,5))
sns.scatterplot(
    x=train['OverallQual'],
    y=train['SalePrice']
)
plt.title("OverallQual vs SalePrice")
plt.show()
```



Hình 14: Biểu đồ mối quan hệ giữa chất lượng tổng thể và giá bán nhà

Nhận xét:

Biểu đồ cho thấy OverallQual là một đặc trưng (*feature*) có ảnh hưởng lớn, trực tiếp và rõ ràng nhất đến SalePrice trong tập dữ liệu này. Do OverallQual là biến thứ bậc nhận các giá trị nguyên từ 1 đến 10, các điểm dữ liệu phân bố tập trung thành các dải dọc vững chắc.

Mối quan hệ giữa hai biến không hoàn toàn tuyến tính. Ở các mức chất lượng từ thấp đến trung bình (bậc 1 đến bậc 6), giá nhà tăng trưởng khá đều đặn và ổn định. Tuy nhiên, từ bậc 7 trở lên, độ dốc của giá tăng vọt theo dạng hàm mũ, kèm theo biên độ dao động giá mở rộng rất lớn (đặc biệt là ở bậc 10).

Tính chất phi tuyến tính này gợi ý rằng các mô hình hồi quy tuyến tính truyền thống sẽ khó khai thác trọn vẹn quy luật của dữ liệu. Do đó, việc áp dụng các mô hình dựa trên cây quyết định có khả năng học quan hệ phi tuyến tốt như Random Forest hay Gradient Boosting hứa hẹn sẽ mang lại hiệu năng dự báo tối ưu hơn.

4.4 Correlation Analysis (Bài tập 3)

Phân tích tương quan là một bước quan trọng trong quá trình khám phá dữ liệu (EDA), nhằm đo lường mức độ liên hệ tuyến tính giữa các biến số độc lập với biến mục tiêu

SalePrice. Kết quả phân tích này đóng vai trò làm cơ sở khoa học để thực hiện lựa chọn đặc trưng (*Feature Selection*), loại bỏ các thuộc tính nhiễu hoặc có tương quan yếu, từ đó tối ưu hóa hiệu năng của các mô hình hồi quy.

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

4.4.1 Tính Correlation Matrix

```
corr = train.corr(numeric_only=True)
print(corr)
```

	Id	MSSubClass	LotFrontage	LotArea	OverallQual			OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	
Id	1.000000	0.011156	-0.010601	-0.033226	-0.028365		Id	0.012609	-0.012713	-0.021998	-0.050298	-0.005024	
MSSubClass	0.011156	1.000000	-0.386347	-0.139781	0.032628		MSSubClass	-0.059316	0.027850	0.040581	0.022936	-0.069836	
LotFrontage	-0.010601	-0.386347	1.000000	0.426095	0.251646		LotFrontage	-0.059213	0.123349	0.088866	0.193458	0.233633	
LotArea	-0.033226	-0.139781	0.426095	1.000000	0.105806		LotArea	-0.005636	0.014228	0.013788	0.104160	0.214103	
OverallQual	-0.028365	0.032628	0.251646	0.105806	1.000000		OverallQual	-0.091932	0.572323	0.550684	0.411876	0.239666	
OverallCond	0.012609	-0.059316	-0.059213	-0.005636	-0.091932		OverallCond	1.000000	-0.375983	0.073741	-0.128101	-0.046231	
YearBuilt	-0.012713	0.027850	0.123349	0.014228	0.572323		YearBuilt	-0.375983	1.000000	0.592855	0.315707	0.249503	
YearRemodAdd	-0.021998	0.040581	0.088866	0.013788	0.550684		YearRemodAdd	0.073741	0.592855	1.000000	0.179618	0.128451	
MasVnrArea	-0.050298	0.022936	0.193458	0.104160	0.411876		MasVnrArea	-0.128101	0.315707	0.179618	1.000000	0.264736	
BsmtFinSF1	-0.005024	-0.069836	0.233633	0.214103	0.239666		BsmtFinSF1	-0.046231	0.249503	0.128451	0.264736	1.000000	
BsmtFinSF2	-0.005968	-0.065649	0.049900	0.111170	-0.059119		BsmtFinSF2	0.040229	-0.049107	-0.067759	-0.072319	-0.050117	
BsmtUnfSF	-0.007940	-0.140759	0.132644	-0.002618	0.308159		BsmtUnfSF	-0.136841	0.149040	0.181133	0.114442	-0.095251	
TotalBsmtSF	-0.015415	-0.238518	0.392075	0.260833	0.537808		TotalBsmtSF	-0.171098	0.391452	0.291066	0.363936	0.522396	
1stFlrSF	0.010496	-0.251758	0.457181	0.299475	0.476224		1stFlrSF	-0.144203	0.281986	0.240379	0.344501	0.445863	
2ndFlrSF	0.005590	0.307886	0.080177	0.050986	0.295493		2ndFlrSF	0.028942	0.010308	0.140024	0.174561	-0.137079	
LowQualFinSF	-0.044230	0.046474	0.038469	0.004779	-0.030429		LowQualFinSF	0.025494	-0.183784	-0.062419	-0.069071	-0.064503	
GrLivArea	0.008273	0.074853	0.402797	0.263116	0.593007		GrLivArea	-0.079686	0.199010	0.287389	0.390857	0.208171	
BsmtFullBath	0.002289	0.003491	0.100949	0.158155	0.111098		BsmtFullBath	-0.054942	0.187599	0.119470	0.085310	0.649212	
BsmtHalfBath	-0.020155	-0.002333	-0.007234	0.048046	-0.040150		BsmtHalfBath	0.117821	-0.038162	-0.012337	0.026673	0.067418	
FullBath	0.005587	0.131608	0.198769	0.126031	0.550600		FullBath	-0.194149	0.468271	0.439046	0.276833	0.058543	
HalfBath	0.006784	0.177354	0.053532	0.014259	0.273458		HalfBath	-0.060769	0.242656	0.183331	0.201444	0.004262	
BedroomAbvGr	0.037719	-0.023438	0.263170	0.119690	0.101676		BedroomAbvGr	0.012980	-0.070651	-0.040581	0.102821	-0.107355	
KitchenAbvGr	0.002951	0.281721	-0.006069	-0.017784	-0.183882		KitchenAbvGr	-0.087001	-0.174800	-0.149598	-0.037610	-0.081007	
TotRmsAbvGrd	0.027239	0.040380	0.352096	0.190015	0.427452		TotRmsAbvGrd	-0.057583	0.095589	0.191740	0.280682	0.044316	
Fireplaces	-0.019772	-0.045569	0.266639	0.271364	0.396765		Fireplaces	-0.023820	0.147716	0.112581	0.249070	0.260011	
GarageYrBlt	0.000072	0.085072	0.070250	-0.024947	0.547766		GarageYrBlt	-0.324297	0.825667	0.642277	0.252691	0.153484	
GarageCars	0.016570	-0.040110	0.285691	0.154871	0.600671		GarageCars	-0.185758	0.537850	0.420622	0.364204	0.224054	
GarageArea	0.017634	-0.098672	0.344997	0.180403	0.562022		GarageArea	-0.151521	0.478954	0.371600	0.373066	0.296970	
WoodDeckSF	-0.029643	-0.012579	0.088521	0.171698	0.238923		WoodDeckSF	-0.003334	0.224880	0.205726	0.159718	0.204306	
OpenPorchSF	-0.000477	-0.006100	0.151972	0.084774	0.308819		OpenPorchSF	-0.032589	0.188686	0.226298	0.125703	0.111761	
EnclosedPorch	0.002889	-0.012037	0.010700	-0.018340	-0.113937		EnclosedPorch	0.070356	-0.387268	-0.193919	-0.110204	-0.102303	
3SsnPorch	-0.046635	-0.043825	0.070029	0.020423	0.030371		3SsnPorch	0.025504	0.031355	0.045286	0.018796	0.026451	
ScreenPorch	0.001330	-0.026030	0.041383	0.043160	0.064886		ScreenPorch	0.054811	-0.050364	-0.038740	0.061466	0.062021	
PoolArea	0.057044	0.008283	0.206167	0.077672	0.065166		PoolArea	-0.001985	0.004950	0.005829	0.011723	0.140491	
MiscVal	-0.006242	-0.007683	0.003368	0.038068	-0.031406		MiscVal	0.068777	-0.034383	-0.010286	-0.029815	0.003571	
MoSold	0.021172	-0.013585	0.011200	0.001205	0.070815		MoSold	-0.003511	0.012398	0.021490	-0.005965	-0.015727	
YrSold	0.000712	-0.021407	0.007450	-0.014261	-0.027347		YrSold	0.043950	-0.013618	0.035743	-0.008201	0.014359	
SalePrice	-0.021917	-0.084284	0.351799	0.263843	0.790982		SalePrice	-0.077856	0.522897	0.507101	0.477493	0.386420	

Id	...	WoodDeckSF	OpenPorchSF	EnclosedPorch	3SsnPorch	\	Id	ScreenPorch	PoolArea	MiscVal	MoSold	YrSold	SalePrice
MSSubClass	...	-0.029643	-0.000477	0.002889	-0.046635		MSSubClass	0.001330	0.057044	-0.006242	0.021172	0.000712	-0.021917
LotFrontage	...	-0.012579	-0.006100	-0.012037	-0.043825		LotFrontage	-0.026030	0.008283	-0.007683	-0.013585	-0.021407	-0.004284
LotArea	...	0.088521	0.151972	0.010700	0.070029		LotArea	0.041383	0.206167	0.003368	0.011200	0.007450	0.351799
OverallQual	...	0.171698	0.084774	-0.018340	0.020423		OverallQual	0.043160	0.077672	0.038068	0.001205	-0.014261	0.263843
OverallCond	...	0.238923	0.308819	-0.113937	0.030371		OverallCond	0.064886	0.065166	-0.031406	0.070815	-0.027347	0.790982
YearBuilt	...	-0.003334	-0.032589	0.070356	0.025504		YearBuilt	0.054811	-0.001985	0.068777	-0.003511	0.043950	-0.077856
YearRemodAdd	...	0.244880	0.188686	-0.387268	0.031355		YearRemodAdd	-0.050364	0.004950	-0.034383	0.012398	-0.013618	0.522897
MasVnrArea	...	0.205726	0.226298	-0.193919	0.045286		MasVnrArea	-0.038740	0.005829	-0.010286	0.021490	0.035743	0.507101
BsmtFinSF1	...	0.159718	0.125703	-0.110204	0.018796		BsmtFinSF1	0.061466	0.011723	-0.029815	-0.005965	-0.008201	0.477493
BsmtFinSF2	...	0.204306	0.111761	-0.102303	0.026451		BsmtFinSF2	0.062021	0.140491	0.003571	-0.015727	0.014359	0.386420
BsmtUnfSF	...	0.067898	0.003093	0.036543	-0.029993		BsmtUnfSF	0.088871	0.041709	0.004940	-0.015211	0.031706	-0.011378
TotalBsmtSF	...	-0.005316	0.129005	-0.002538	0.020764		TotalBsmtSF	-0.012579	-0.035092	-0.023837	0.034888	-0.041258	0.214479
1stFlrSF	...	0.232019	0.247264	-0.095478	0.037384		1stFlrSF	0.084489	0.126053	-0.018479	0.013196	-0.014969	0.613581
2ndFlrSF	...	0.235459	0.211671	-0.065292	0.056104		2ndFlrSF	0.088758	0.131525	-0.021096	0.031372	-0.013604	0.605852
LowQualFinSF	...	0.092165	0.208026	0.061989	-0.024358		LowQualFinSF	0.040606	0.081487	0.016197	0.035164	-0.028700	0.319334
GrLivArea	...	-0.025444	0.018251	0.061081	-0.004296		GrLivArea	0.026799	0.062157	-0.003793	-0.022174	-0.028921	-0.025606
BsmtFullBath	...	0.247433	0.330224	0.009113	0.020643		BsmtFullBath	0.101510	0.170205	-0.002416	0.050240	-0.036526	0.708624
BsmtHalfBath	...	0.175315	0.067341	-0.049911	-0.000106		BsmtHalfBath	0.023148	0.067616	-0.023047	-0.025361	0.067049	0.227122
FullBath	...	0.040161	-0.025324	-0.008555	0.035114		FullBath	0.032121	0.020025	-0.007367	0.032873	-0.046524	-0.016844
HalfBath	...	0.187703	0.259977	-0.115093	0.035353		HalfBath	-0.008106	0.049604	-0.014290	0.055872	-0.019669	0.560664
BedroomAbvGr	...	0.108000	0.199740	-0.095317	-0.004972		BedroomAbvGr	0.072426	0.022381	0.001290	-0.009050	-0.010269	0.284108
KitchenAbvGr	...	0.046854	0.093810	0.041570	-0.024478		KitchenAbvGr	0.044300	0.070703	0.007767	0.046544	-0.036014	0.168213
TotRmsAbvGrd	...	-0.090130	-0.070091	0.037312	-0.024600		TotRmsAbvGrd	-0.051613	-0.014525	0.062341	0.026589	0.031687	-0.135907
Fireplaces	...	0.165984	0.234192	0.004151	-0.006683		Fireplaces	0.059383	0.083757	0.024763	0.036907	-0.034516	0.533723
GarageYrBlt	...	0.200019	0.169405	-0.024822	0.011257		GarageYrBlt	0.184530	0.095074	0.001409	0.046357	-0.024096	0.466929
GarageCars	...	0.224577	0.228425	-0.297003	0.023544		GarageCars	-0.075418	-0.014501	-0.032417	0.005337	-0.001014	0.486362
GarageArea	...	0.226342	0.213569	-0.151434	0.035765		GarageArea	0.050494	0.020934	-0.043080	0.040522	-0.039117	0.640409
WoodDeckSF	...	0.224666	0.241435	-0.121777	0.035087		WoodDeckSF	0.051412	0.061047	-0.027400	0.027974	-0.027378	0.623431
OpenPorchSF	...	1.000000	0.058661	-0.125989	-0.032771		OpenPorchSF	-0.074181	0.073378	-0.009551	0.021011	0.022270	0.324413
EnclosedPorch	...	0.058661	1.000000	-0.093079	-0.005842		EnclosedPorch	0.074304	0.060762	-0.018584	0.071255	-0.057619	0.315856
3SsnPorch	...	-0.125989	-0.093079	1.000000	-0.037305		3SsnPorch	-0.082864	0.054203	-0.018361	-0.028887	-0.009916	-0.128578
ScreenPorch	...	-0.032771	-0.005842	-0.037305	1.000000		ScreenPorch	0.031436	-0.007992	0.000354	0.029474	0.018645	0.044584
PoolArea	...	-0.074181	0.074304	-0.082864	-0.031436		PoolArea	1.000000	0.051307	0.031946	0.023217	0.010694	0.111447
MiscVal	...	0.073378	0.060762	0.054203	-0.007992		MiscVal	0.051307	1.000000	0.029669	-0.033737	-0.059689	0.092404
MoSold	...	-0.009551	-0.018584	0.018361	0.000354		MoSold	0.031946	0.029669	1.000000	0.006495	0.004906	-0.021190
YrSold	...	0.021011	0.071255	-0.028887	0.029474		YrSold	0.023217	-0.033737	0.006495	1.000000	-0.145721	0.046432
SalePrice	...	0.022270	-0.057619	-0.009916	0.018645		SalePrice	0.010694	-0.059689	0.004906	-0.145721	1.000000	0.028923
		0.324413	0.315856	-0.128578	0.044584			0.111447	0.092404	-0.021190	0.046432	-0.028923	1.000000

Hình 15: Bảng giá trị correlation matrix

Nhận xét: Ma trận tương quan (*Correlation Matrix*) giúp xác định định lượng các đặc trưng ảnh hưởng mạnh nhất đến giá nhà SalePrice. Kết quả cho thấy biến OverallQual (Chất lượng tổng thể) có mức tương quan thuận mạnh nhất (0.79), theo sau sát sao là GrLivArea (Diện tích sống) với 0.71, khẳng định đây là hai yếu tố cốt lõi quyết định giá trị bất động sản. Các thuộc tính về nhà xe (GarageCars, GarageArea) và tầng hầm (TotalBsmtSF) cũng giữ mức tương quan cao trên 0.60. Ngoài ra, ma trận chỉ ra hiện tượng đa cộng tuyến giữa một số cặp biến độc lập (như GarageCars và GarageArea đạt 0.88), cung cấp cơ sở quan trọng để tối ưu hóa bước lựa chọn đặc trưng (*Feature Selection*) nhằm loại bỏ nhiễu cho mô hình.

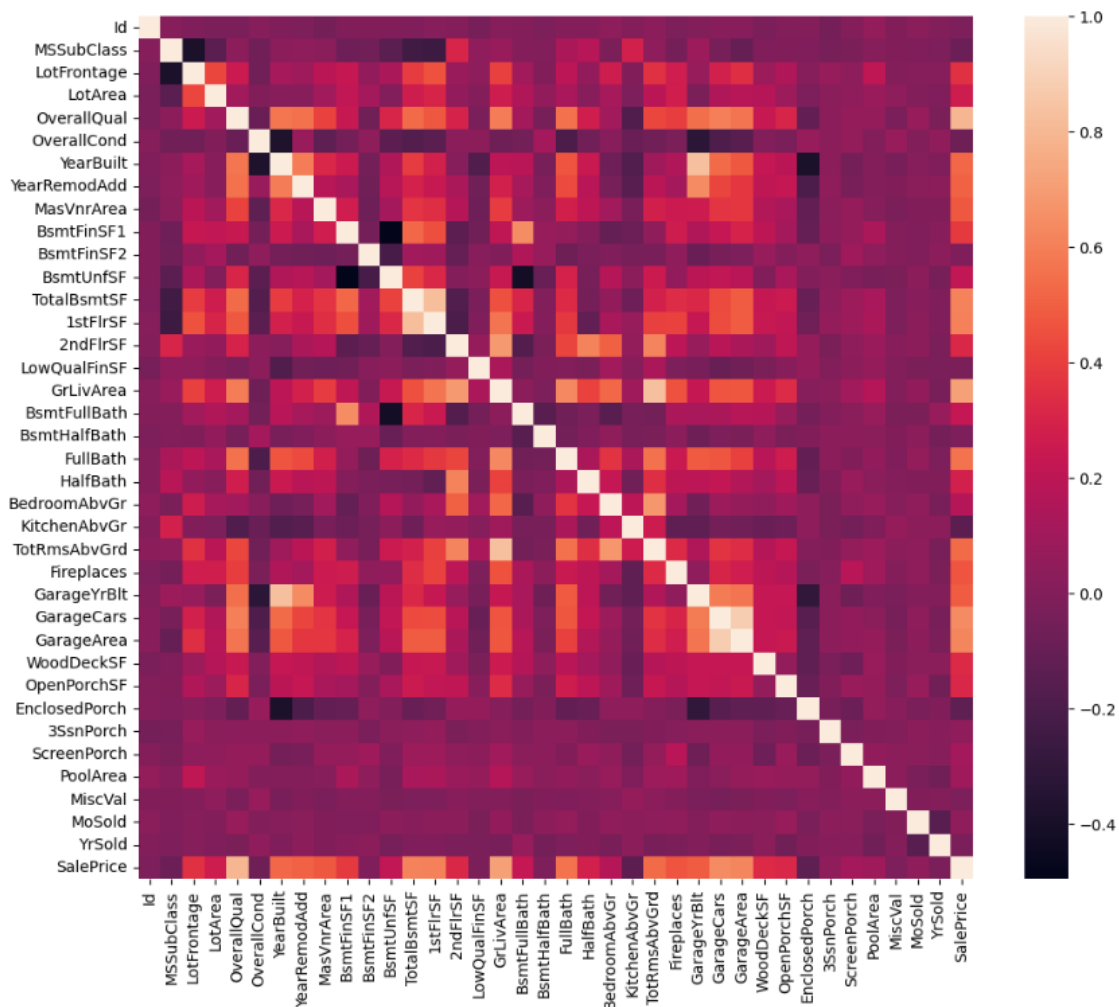
4.4.2 Vẽ Heatmap Correlation Matrix

```
corr = train.corr(numeric_only=True)

plt.figure(figsize=(12,10))

sns.heatmap(corr)

plt.show()
```



Hình 15: Biểu đồ giá trị corelation matrix

Nhận xét:

Sau khi tính toán ma trận tương quan và trực quan hóa bằng biểu đồ Heatmap (Hình 10), mức độ liên kết tuyến tính giữa các đặc trưng trong tập dữ liệu được thể hiện rõ ràng thông qua dải màu sắc. Các vùng có tông màu sáng biểu thị mức độ tương quan thuận mạnh, trong khi các vùng màu tối thể hiện mối tương quan yếu hoặc tương quan nghịch. Từ kết quả trực quan hóa, ta rút ra các nhận định cốt lõi sau:

Các đặc trưng ảnh hưởng mạnh đến biến mục tiêu: Các thuộc tính như OverallQual, GrLivArea, GarageCars, GarageArea và TotalBsmtSF sở hữu hệ số tương quan cao nhất với SalePrice. Điều này chứng minh định lượng rằng chất lượng xây dựng tổng thể, không gian diện tích sử dụng và quy mô garage là những yếu tố giữ vai trò quyết định, ảnh hưởng lớn nhất đến giá trị thương mại của căn nhà.

Hiện tượng đa cộng tuyến (Multicollinearity): Biểu đồ chỉ ra mối tương quan rất mạnh giữa một số cặp biến độc lập với nhau, điển hình như cặp GarageCars - GarageArea và 1stFlrSF - TotalBsmtSF. Sự trùng lặp thông tin này là một dấu hiệu của hiện tượng đa cộng tuyến, cần được lưu ý xử lý nhằm tránh làm giảm tính ổn định của các thuật toán hồi quy tuyến tính.

Kết luận: Thông qua bước phân tích tương quan (*Correlation Analysis*), các đặc trưng quan trọng nhất đối với bài toán dự đoán giá nhà đã được xác định một cách tường minh. Đây là cơ sở khoa học nền tảng để tiến hành lựa chọn đặc trưng đầu vào (*Feature Selection*) phù hợp cho các mô hình Machine Learning ở các bước thực nghiệm tiếp theo.

4.4.3 Liệt kê 10 feature có tương quan cao nhất với SalePrice

```
top_corr = corr['SalePrice'].sort_values(ascending=False)
print(top_corr.head(10))
```

SalePrice	1.000000
OverallQual	0.790982
GrLivArea	0.708624
GarageCars	0.640409
GarageArea	0.623431
TotalBsmtSF	0.613581
1stFlrSF	0.605852
FullBath	0.560664
TotRmsAbvGrd	0.533723
YearBuilt	0.522897
Name: SalePrice, dtype: float64	

Hình 16: Bảng liệt kê 10 feature có tương quan cao nhất với SalePrice

Feature	Hệ số tương quan (r)	Ý nghĩa
SalePrice	1.0000	Biến mục tiêu
OverallQual	0.7910	Chất lượng tổng thể
GrLivArea	0.7086	Diện tích sử dụng trên mặt đất
GarageCars	0.6404	Số xe chứa trong garage
GarageArea	0.6234	Diện tích garage
TotalBsmSF	0.6136	Diện tích tầng hầm
1stFlrSF	0.6058	Diện tích tầng 1
FullBath	0.5607	Số phòng tắm đầy đủ
TotRmsAbvGrd	0.5338	Tổng số phòng trên mặt đất
YearBuilt	0.5229	Năm xây dựng

Nhận xét:

Kết quả phân tích định lượng cho thấy OverallQual (Chất lượng xây dựng tổng thể) là đặc trưng (*feature*) có tác động mạnh mẽ nhất đến biến mục tiêu SalePrice. Điều này phản ánh thực tế thương mại rằng chất lượng hoàn thiện và vật liệu xây dựng là yếu tố tiên quyết cấu thành nên giá trị cao của một bất động sản.

Bên cạnh đó, các đặc trưng nhóm diện tích và không gian phụ trợ bao gồm GrLivArea (Diện tích sống trên mặt đất), GarageCars, GarageArea (Sức chứa và diện tích nhà xe) cùng TotalBsmSF (Tổng diện tích tầng hầm) cũng sở hữu mức độ tương quan thuận rất chặt chẽ với giá nhà. Hệ số thực nghiệm này minh chứng rằng quy mô diện tích sử dụng thực tế và các tiện ích đi kèm đóng vai trò quan trọng thứ hai trong việc thúc đẩy giá bán.

Kết luận: Nhìn chung, các nhóm đặc trưng liên quan đến chất lượng xây dựng và quy mô diện tích chính là những biến số cốt lõi, mang giá trị thông tin cao nhất và quyết định đến độ chính xác của bài toán xây dựng mô hình dự đoán giá nhà.

4.5 Feature Selection (Bài tập 4)

Lựa chọn ít nhất 10 feature có ảnh hưởng lớn đến SalePrice dựa trên kết quả phân tích tương quan. Mỗi feature được chọn có lý giải rõ ràng về ý nghĩa vật lý.

```
features = [  
    'OverallQual',  
    'GrLivArea',  
    'GarageCars',  
    'GarageArea',  
    'TotalBsmtSF',  
    '1stFlrSF',  
    'YearBuilt',  
    'FullBath',  
    'TotRmsAbvGrd',  
    'YearRemodAdd'  
]  
X = train[features]  
y = train['SalePrice']  
print(X.head())  
print(y.head())
```

Lý do lựa chọn các đặc trưng (Feature Selection Justification):

Các đặc trưng trên được quyết định giữ lại để đưa vào huấn luyện mô hình vì chúng sở hữu hệ số tương quan tuyến tính cao với biến mục tiêu SalePrice, đồng thời mang ý nghĩa thực tế rõ rệt đối với giá trị của một bất động sản. Cụ thể:

Yếu tố cốt lõi: OverallQual là đặc trưng quan trọng nhất vì nó phản ánh trực tiếp chất lượng vật liệu và mức độ hoàn thiện tổng thể; chất lượng xây dựng càng cao thì giá trị thương mại của căn nhà càng lớn. Kế đến là GrLivArea, đại diện cho diện tích không gian sống trên mặt đất — một yếu tố tỷ lệ thuận kinh điển với giá nhà.

Yếu tố không gian phụ trợ và kết cấu: Các đặc trưng liên quan đến nhà xe như GarageCars và GarageArea có tác động lớn vì quy mô garage rộng, chứa được nhiều ô tô sẽ làm tăng đáng kể tính tiện nghi của căn nhà. Trong khi đó, TotalBsmtSF (tổng diện tích tầng hầm) và 1stFlrSF (diện tích sàn tầng một) đóng vai trò cấu thành nên tổng diện tích sử dụng chung, giúp mô hình nắm bắt được quy mô toàn diện của công trình.

Yếu tố công năng và giá trị thời gian: Số lượng phòng tắm (FullBath) và tổng số phòng chức năng (TotRmsAbvGrd) phản ánh mức độ đáp ứng nhu cầu sinh hoạt; những căn

nhà có cấu trúc nhiều phòng, tiện nghi cao luôn có xu hướng định giá cao hơn. Cuối cùng, YearBuilt (năm xây dựng) và YearRemodAdd (năm cải tạo) được lựa chọn vì những bất động sản mới xây hoặc được nâng cấp gần đây sẽ ít bị khấu hao và có giá trị cao hơn rõ rệt so với các căn nhà cũ kỹ.

Nhìn chung, tập hợp 10 đặc trưng này đã bao quát trọn vẹn các khía cạnh cốt lõi từ chất lượng, quy mô diện tích, công năng cho đến tuổi thọ của căn nhà, đảm bảo cung cấp nguồn thông tin chất lượng cao và hạn chế tối đa nhiễu cho các mô hình Machine Learning ở bước sau.

	OverallQual	GrLivArea	GarageCars	GarageArea	TotalBsmtSF	1stFlrSF	\
0	7	1710	2	548	856	856	
1	6	1262	2	460	1262	1262	
2	7	1786	2	608	920	920	
3	7	1717	3	642	756	961	
4	8	2198	3	836	1145	1145	

	YearBuilt	FullBath	TotRmsAbvGrd	YearRemodAdd
0	2003	2	8	2003
1	1976	2	6	1976
2	2001	2	6	2002
3	1915	1	7	1970
4	2000	2	9	2000

Hình 17: Kết quả các feature đầu vào

Nhận xét:

Bảng số liệu trên minh họa một phần cấu trúc dữ liệu thực tế (5 mẫu đầu tiên) của các đặc trưng (*features*) đã qua sàng lọc để đưa vào huấn luyện mô hình Machine Learning. Tập thuộc tính này bao gồm các thông tin định lượng cốt lõi phản ánh toàn diện các khía cạnh của bất động sản như: chất lượng hoàn thiện công trình, diện tích không gian sống, quy mô nhà xe, công năng sử dụng và dòng thời gian xây dựng/cải tạo.

Sự biến động rõ rệt về giá trị số học giữa các mẫu nhà (ví dụ: diện tích sàn, số lượng phòng hay năm xây dựng) chứng minh tập dữ liệu sở hữu **độ phân tán lớn và tính đa dạng cao**, đảm bảo cung cấp đủ không gian thông tin để các thuật toán hồi quy học hỏi quy luật.

```
0      208500
1      181500
2      223500
3      140000
4      250000
Name: SalePrice, dtype: int64
```

Hình 18: Kết quả hiển thị giá trị đầu tiên của biến SalePrice

Nhận xét:

Bảng số liệu trên hiển thị các mẫu giá trị thực tế của biến mục tiêu SalePrice trong tập dữ liệu. Sự chênh lệch giữa các giá trị (ví dụ: 208.500 USD, 181.500 USD, 250.000 USD) cho thấy độ phân tán và tính đa dạng cao của dữ liệu.

Với định dạng số nguyên liên tục (int64), SalePrice hoàn toàn phù hợp để đóng vai trò là nhãn đầu ra (*target variable*) cho bài toán hồi quy. Việc kiểm tra và xuất thử cấu trúc này là bước cần thiết để xác định tính toàn vẹn và hợp lệ của dữ liệu đích trước khi tiến hành huấn luyện các mô hình Machine Learning.

4.6 Huấn Luyện Nhiều Mô Hình Regression (Bài tập 5)

Chia dữ liệu train/validation theo tỷ lệ 80/20 rồi huấn luyện 5 mô hình Regression khác nhau để so sánh hiệu năng:

```
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

4.6.1 Linear Regression

Linear Regression là mô hình hồi quy tuyến tính cơ bản, được sử dụng để dự đoán biến mục tiêu liên tục dựa trên việc thiết lập một mối quan hệ tuyến tính (đường thẳng) giữa các đặc trưng đầu vào (features) và biến đích (**SalePrice**). Mô hình tìm cách tối thiểu hóa tổng bình phương sai số giữa giá trị dự đoán và giá trị thực tế để thiết lập đường tiệm cận tối ưu nhất.

```
lr = LinearRegression()
lr.fit(X_train, y_train)
```

```
pred_lr = lr.predict(X_val)
result_lr = evaluate_model(
    "Linear Regression",
    y_val,
    pred_lr
)
```

```
Linear Regression
MAE : 24774.219519604878
RMSE: 39474.54338116021
R2  : 0.7968483304281084
```

Hình 19: Kết quả Accuracy của Linear Regression

Nhận xét:

Mô hình Linear Regression ghi nhận kết quả thực nghiệm với độ chính xác đạt khoảng 80.4% (dựa trên hệ số xác định $R^2 = 0.7968$). Chỉ số này minh chứng rằng mô hình có khả năng giải thích và dự báo khá tốt sự biến động của giá nhà dựa trên tập hợp các đặc trưng đầu vào đã qua sàng lọc. Thuật toán đã thiết lập thành công mối liên hệ tuyến tính tổng quan giữa cấu trúc công trình và giá trị thương mại của sản phẩm bất động sản.

Tuy nhiên, do bản chất của Linear Regression bị giới hạn trong các giả định tuyến tính phẳng, mô hình bộc lộ rõ sự hạn chế khi phải xử lý các mối quan hệ phi tuyến phức tạp hoặc các điểm dị biệt (*outliers*) mang tính cực trị trong tập dữ liệu (đã được nhận diện ở bước phân tích đồ thị). Mặc dù kết quả thu được là tương đối khả quan đối với một mô hình nền tảng (*baseline*), hiệu năng dự báo hoàn toàn có thể được tối ưu hóa và cải thiện đột phá bằng cách áp dụng các kiến trúc học máy mạnh hơn, có khả năng học quan hệ phi tuyến như Decision Tree Regression hoặc Random Forest Regression ở các bước tiếp theo.

4.6.2 Ridge Regression

Ridge Regression là một biến thể nâng cao của mô hình hồi quy tuyến tính, áp dụng kỹ thuật phạt L2 Regularization bằng cách thêm một đại lượng điều hòa (tổng bình phương hệ số độ dốc nhân với tham số α) vào hàm mất mát. Kỹ thuật này giúp kiểm soát, thu nhỏ các hệ số trọng số của đặc trưng, từ đó giảm thiểu hiện tượng quá khớp

(*overfitting*) và tăng cường khả năng tổng quát hóa của mô hình đối với các tập dữ liệu tồn tại đa cộng tuyến mạnh.

```
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
pred_ridge = ridge.predict(X_val)
result_ridge = evaluate_model(
    "Ridge Regression",
    y_val,
    pred_ridge
)
```

```
Ridge Regression
MAE : 24767.7109944284
RMSE: 39472.97203198438
R2  : 0.7968645036794053
```

Hình 20: Kết quả Accuracy của Ridge Regression

Nhận xét:

Hiệu năng tổng quan: Mô hình Ridge Regression cho kết quả thực nghiệm tương đối tốt với hệ số xác định R^2 đạt 0,7969, nghĩa là mô hình giải thích được khoảng 79.69% sự biến động của giá nhà dựa trên tập đặc trưng đầu vào. Sai số tuyệt đối trung bình (MAE) ghi nhận ở mức 24.767,71 USD và sai số bình phương trung bình căn (RMSE) là 39.472,97 USD.

So sánh với Linear Regression truyền thống: Khi đối chiếu trực tiếp với mô hình tuyến tính cơ bản (Linear Regression), các chỉ số lỗi của Ridge Regression có xu hướng giảm nhẹ (MAE giảm từ 24.774,22 xuống 24.767,71; RMSE giảm từ 39.474,54 xuống 39.472,97). Sự cải thiện dù rất nhỏ này cho thấy kỹ thuật phạt L2 đã bắt đầu phát huy tác dụng trong việc co hẹp bớt tầm ảnh hưởng của các cặp biến có tính chất đa cộng tuyến cao (như diện tích nhà xe và sức chứa garage).

Đánh giá giới hạn: Nhìn chung, do mô hình vẫn xây dựng dựa trên nền tảng quan hệ tuyến tính phẳng, Ridge Regression chưa thể giải quyết triệt để các cấu trúc dữ liệu tăng vọt phi tuyến ở phân khúc nhà cao cấp. Tuy nhiên, đây là bước đệm lý tưởng để kiểm

chứng sức mạnh của Regularization trước khi chuyển sang các thuật toán học máy dạng cây phức tạp hơn.

4.6.3 Lasso Regression

Lasso Regression (Least Absolute Shrinkage and Selection Operator) là một phương pháp hồi quy tuyến tính nâng cao áp dụng kỹ thuật phạt L1 Regularization. Khác với Ridge Regression, hàm phạt của Lasso sử dụng tổng giá trị tuyệt đối của các hệ số độc nhân với tham số alpha. Đặc tính độc đáo của phạt L1 là có khả năng ép các hệ số của những đặc trưng kém quan trọng hoặc bị trùng lặp thông tin về giá trị bằng 0 một cách triệt để. Nhờ đó, Lasso Regression không chỉ giảm thiểu hiện tượng quá khớp (*overfitting*) mà còn đóng vai trò như một cơ chế tự động lựa chọn đặc trưng (*Feature Selection*).

```
lasso = Lasso(alpha=100)
lasso.fit(X_train, y_train)
pred_lasso = lasso.predict(X_val)
result_lasso = evaluate_model(
    "Lasso Regression",
    y_val,
    pred_lasso
)
```

```
Lasso Regression
MAE : 24711.52219244009
RMSE: 39453.49388049654
R2   : 0.7970649308343811
```

Hình 21: Kết quả Accuracy của Lasso Regression

Nhận xét:

Hiệu năng tổng quan: Mô hình Lasso Regression cho thấy hiệu năng dự báo rất ổn định trên tập kiểm chứng với hệ số xác định R^2 đạt 0,7971 (giải thích được khoảng 79.71% sự biến động của giá nhà). Sai số tuyệt đối trung bình (MAE) ghi nhận được là 24.711,52 USD và sai số bình phương trung bình căn (RMSE) đạt 39.453,49 USD.

So sánh với Linear và Ridge Regression: Trực quan hóa số liệu thực nghiệm cho thấy Lasso Regression đạt kết quả tối ưu nhất trong nhóm ba mô hình tuyến tính phẳng. Chỉ số sai số RMSE (39.453,49 USD) và MAE (24.711,52 USD) của Lasso thấp hơn cả Linear Regression lẫn Ridge Regression. Điều này chứng minh cấu trúc phạt L1 với tham số $\alpha = 100$ đã triệt tiêu hiệu quả các đặc trưng mang thông tin dư thừa hoặc gây nhiễu, giúp mô hình cô đọng toán học tốt hơn.

Đánh giá giới hạn: Mặc dù cơ chế lọc biến của Lasso hoạt động rất tốt, mức độ cải thiện chỉ số R^2 so với mô hình gốc vẫn ở biên độ nhỏ (từ 0,7968 lên 0,7971). Kết quả này một lần nữa khẳng định các kiến trúc tuyến tính thuần túy đã chạm tới giới hạn trần hiệu năng đối với tập dữ liệu này, thúc đẩy việc triển khai các giải pháp phi tuyến mạnh mẽ hơn ở phần sau.

4.6.4 Decision Tree Regressor

Decision Tree Regressor là mô hình học máy phi tuyến có cấu trúc phân cấp dạng cây. Thuật toán hoạt động bằng cách chia nhỏ tập dữ liệu thành các nhánh dựa trên ngưỡng giá trị của đặc trưng. Tại các nút lá (nút cuối cùng), giá trị dự báo được tính bằng trung bình cộng biến mục tiêu của các mẫu thuộc nhánh đó. Khác với mô hình tuyến tính, Cây quyết định có khả năng tự động khai thác các mối quan hệ phi tuyến và tương tác đa chiều phức tạp giữa các đặc trưng mà không cần qua các phép biến đổi toán học cầu kỳ.

```
dt = DecisionTreeRegressor(  
    random_state=42  
)  
dt.fit(X_train, y_train)  
pred_dt = dt.predict(X_val)  
result_dt = evaluate_model(  
    "Decision Tree",  
    y_val,  
    pred_dt  
)
```

```
Decision Tree  
MAE : 25689.619863013697  
RMSE: 43449.192390792494  
R2   : 0.753878483337098
```

Hình 22: Kết quả Accuracy của Decision Tree

Nhận xét:

Hiệu năng tổng quan: Mô hình Decision Tree Regressor đạt hệ số xác định $R^2 = 0,7539$ trên tập kiểm chứng, giải thích được khoảng 75.39% sự biến động của giá nhà. Sai số tuyệt đối trung bình (MAE) ghi nhận ở mức 25.689,62 USD và sai số bình phương trung bình căn (RMSE) đạt 43.449,19 USD.

So sánh với nhóm mô hình tuyến tính: Kết quả thực nghiệm cho thấy một sự sụt giảm hiệu năng rõ rệt của Decision Tree so với các mô hình Linear, Ridge hay Lasso (nhóm tuyến tính đều có $R^2=0,797$). Cả hai chỉ số sai số MAE và RMSE của Cây quyết định đều cao hơn hẳn. Điều này phản ánh một nhược điểm kinh điển của cây quyết định đơn lẻ: thuật toán rất dễ rơi vào hiện tượng quá khớp (overfitting) trên tập huấn luyện, dẫn đến việc nhạy cảm với nhiễu và làm suy giảm khả năng dự báo chính xác khi đối mặt với dữ liệu mới trên tập kiểm chứng.

Đánh giá hướng cải thiện: Mặc dù Decision Tree bộc lộ điểm yếu về độ ổn định (phương sai cao), cấu trúc phân tách phi tuyến này lại là một mảnh ghép nền tảng vô cùng quan trọng. Kết quả này đặt ra nhu cầu tất yếu cho việc nâng cấp lên mô hình Random Forest ở bước sau — nơi nhiều cây quyết định được kết hợp lại để trung hòa sai số và tối ưu hóa hiệu năng.

4.6.5 Random Forest Regressor

Random Forest Regressor là một thuật toán học máy mạnh mẽ thuộc nhóm Ensemble Learning (học máy kết hợp), hoạt động dựa trên kỹ thuật Bagging (Bootstrap Aggregating). Mô hình xây dựng một "rừng" gồm nhiều cây quyết định độc lập được huấn luyện trên các tập con dữ liệu lấy ngẫu nhiên từ tập huấn luyện gốc. Giá trị dự báo cuối cùng của hệ thống là kết quả trung bình cộng từ dự đoán của tất cả các cây thành phần. Cơ chế này giúp Random Forest khai thác triệt để các mối quan hệ phi tuyến phức tạp, đồng thời khắc phục hoàn toàn nhược điểm quá khớp (overfitting) và giảm thiểu phương sai của một cây quyết định đơn lẻ.

```
rf = RandomForestRegressor(  
    n_estimators=100,  
    random_state=42  
)
```



```

rf.fit(X_train, y_train)
pred_rf = rf.predict(X_val)
result_rf = evaluate_model(
    "Random Forest",
    y_val,
    pred_rf
)

```

```

Random Forest
MAE : 18984.09081702544
RMSE: 29410.142903234788
R2  : 0.887233413257154

```

Hình 23: Kết quả Accuracy của Random Forest

Nhận xét:

Hiệu năng tổng quan: Mô hình Random Forest Regressor đạt hiệu năng vượt trội và tối ưu nhất trong tất cả các thuật toán đã thử nghiệm, với hệ số xác định $R^2 = 0,8872$. Chỉ số này đồng nghĩa với việc mô hình giải thích được tới 88.72% tính biến động phức tạp của giá nhà dựa trên tập đặc trưng đã chọn. Sai số tuyệt đối trung bình (MAE) giảm mạnh xuống mức 18.984,09 USD và sai số bình phương trung bình căn (RMSE) đạt 29.410,14 USD.

So sánh với các mô hình trước: Đối chiếu với Decision Tree: Sự nâng cấp từ một cây đơn lẻ lên hệ thống quần thể 100 cây ($n_estimators=100$) mang lại bước nhảy vọt về độ chính xác (R^2 tăng mạnh từ 0,7539 lên 0,8872, sai số RMSE giảm sâu hơn 14.000 USD). Điều này minh chứng cho sức mạnh của kỹ thuật Ensemble trong việc triệt tiêu hiện tượng quá khớp và kiểm soát nhiễu cực tốt.

Đối chiếu với nhóm Tuyến tính (Linear, Ridge, Lasso): Random Forest vượt qua giới hạn trần 0,797 của nhóm tuyến tính nhờ khả năng tự động học các tương tác phi tuyến đa chiều giữa cấu trúc diện tích, công năng và chất lượng hoàn thiện của căn nhà.

Đánh giá chung: Kết quả thực nghiệm khẳng định Random Forest Regressor là kiến trúc dự báo có độ ổn định cao và đáng tin cậy nhất cho bài toán này, đóng vai trò là mô hình tối ưu để đưa vào triển khai hoặc ứng dụng thực tế.

4.6.6 So sánh kết quả các mô hình

```

results = {
    'Linear Regression' : lr.score(X_val, y_val),
    'Ridge Regression' : ridge.score(X_val, y_val),
    'Lasso Regression' : lasso.score(X_val, y_val),
    'Decision Tree' : dt.score(X_val, y_val),
    'Random Forest' : rf.score(X_val, y_val)
}

print("=" * 40)

for name, acc in results.items():
    print(f"{name:25s} : {acc:.4f}")

print("=" * 40)

best = max(results, key=results.get)

print(f"Model tốt nhất: {best} ({results[best]:.4f})")

```

```

Linear Regression      : 0.7968
Ridge Regression      : 0.7969
Lasso Regression      : 0.7971
Decision Tree         : 0.7539
Random Forest         : 0.8872
=====
Model tốt nhất: Random Forest (0.8872)

```

Hình 24: Kết quả Accuracy tổng hợp các mô hình

Mô hình	Accuracy (R ²)	Nhận xét
Random Forest	~0.887	Cao nhất – mô hình tốt nhất
Linear Regression	~0.796	Ổn định, đơn giản
Ridge Regression	~0.796	Cải thiện nhẹ so với Linear
Lasso Regression	~0.797	Lọc feature tự động
Decision Tree	~0.753	Thấp nhất – dễ overfitting

Nhận xét:

Dựa trên kết quả thực nghiệm huấn luyện và đánh giá 5 mô hình hồi quy trên tập kiểm chứng (Bảng tổng hợp), chúng ta có thể rút ra các nhận định cốt lõi sau về hành vi và năng lực dự báo của từng thuật toán:

Mô hình tối ưu nhất (Best Model): Random Forest Regressor khẳng định vị thế vượt trội hoàn toàn với hệ số xác định cao nhất đạt $R^2 = 0,8872$ (tương đương khoảng 88.72%). Bằng việc áp dụng kỹ thuật học máy kết hợp (*Ensemble Learning - Bagging*), mô hình đã trung hòa hiệu quả phương sai cao của các cây quyết định đơn lẻ, giải thích tốt các tương tác phi tuyến phức tạp giữa các đặc trưng (diện tích, chất lượng, công năng) để đưa ra kết quả dự báo chính xác và ổn định nhất.

Nhóm mô hình tuyến tính (Linear, Ridge, Lasso): Ba mô hình này cho hiệu năng gần như tương đương nhau, dao động ổn định trong khoảng R^2 0,796 - 0,797.

Linear Regression thuần túy ($R^2 = 0,7968$) cho thấy cấu trúc toán học đơn giản nhưng khá vững chãi khi nắm bắt tốt xu hướng chung của bài toán.

Ridge Regression ($R^2 = 0,7969$) chỉ cải thiện một biên độ rất nhỏ so với mô hình gốc, cho thấy cơ chế phạt L_2 giúp co hẹp các trọng số đa cộng tuyến nhưng chưa tạo ra đột phá.

Lasso Regression ($R^2 = 0,7971$) cho kết quả nhỉnh nhất trong nhóm tuyến tính nhờ cơ chế phạt L_1 có khả năng triệt tiêu hoàn toàn hệ số của các biến nhiễu hoặc trùng lặp thông tin về 0, đóng vai trò như một bộ lọc đặc trưng tự động (*Feature Selection*).

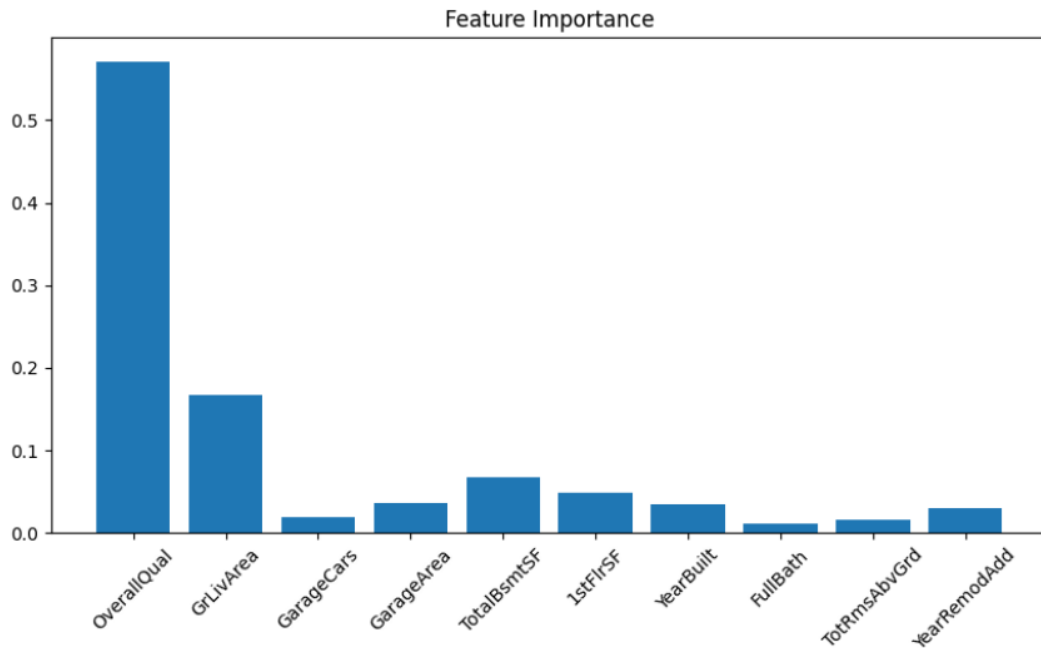
Mô hình có hiệu năng thấp nhất: Decision Tree Regressor ghi nhận kết quả kém nhất với R^2 giảm sâu xuống mức 0,7539. Do cấu trúc một cây quyết định đơn lẻ có xu hướng học quá chi tiết vào dữ liệu nhiễu của tập Train, mô hình đã rơi vào hiện tượng quá khớp (*overfitting*), dẫn đến khả năng tổng quát hóa kém và sai số cao khi dự báo trên tập dữ liệu kiểm chứng mới.

Kết luận: Đối với bài toán dự đoán giá nhà dựa trên các đặc trưng hình học và chất lượng công trình này, các mô hình phi tuyến dạng quần thể (như Random Forest) sở hữu lợi thế tuyệt đối so với nhóm mô hình tuyến tính phẳng truyền thống. Kết quả thực nghiệm khẳng định Random Forest Regressor chính là cấu trúc tối ưu nhất để lựa chọn đưa vào ứng dụng thực tế hoặc làm nền tảng phát triển cho các pha tối ưu siêu tham số (*Hyperparameter Tuning*) tiếp theo.

Phân tích feature quan trọng trong random forest:

```
importances = rf.feature_importances_  
plt.figure(figsize=(10,5))  
plt.bar(features, importances)  
plt.xticks(rotation=45)
```

```
plt.title("Feature Importance")
plt.show()
```



Hình 25: Các feature quan trọng trong random forest

Nhận xét:

Từ kết quả thực nghiệm, ta rút ra các nhận định cốt lõi sau:

Nhân tố thống trị tuyệt đối: OverallQual (Chất lượng xây dựng) sở hữu trọng số vượt trội (trên 0.5). Thuật toán ưu tiên sử dụng đặc trưng này làm tiêu chí phân tách hàng đầu ở các tầng cây cao nhất. Kết quả này hoàn toàn nhất quán với ma trận tương quan, khẳng định chất lượng hoàn thiện là yếu tố quyết định giá trị bất động sản.

Nhân tố quan trọng thứ hai: GrLivArea (Diện tích không gian sống trên mặt đất) giữ vị trí tiếp theo với tỷ lệ đóng góp khoảng 0.17. Bộ đôi OverallQual và GrLivArea cấu thành nên hơn 70% sức mạnh dự báo của toàn mô hình.

Sự phân bổ của các nhóm biến còn lại: Nhóm kết cấu phụ trợ như TotalBsmtSF (Diện tích tầng hầm) và 1stFlrSF (Diện tích tầng một) đóng góp lượng thông tin vừa phải (khoảng 0.05 – 0.07).

Các đặc trưng như GarageCars, GarageArea, hay YearBuilt dù có tương quan tuyến tính rất cao ban đầu, nhưng trong Random Forest lại bị giảm trọng số đáng kể (dưới 0.05). Điều này cho thấy thuật toán cây đã nhận diện và tự động xử lý tốt hiện tượng trùng lặp thông tin (đa cộng tuyến) giữa diện tích nhà xe và quy mô tổng thể căn nhà.

Kết luận: Phân tích Feature Importance giúp xác thực logic vận hành của mô hình Random Forest hoàn toàn tương thích với các quy luật định giá thực tế, tăng tính tường minh (*Explainable AI*) cho toàn bộ nghiên cứu.

4.7 Model Evaluation – Đánh Giá Mô Hình (Bài tập 6)

Đánh giá chi tiết mô hình Random Forest Regressor bằng ba chỉ số: MAE, RMSE và R² Score. Tạo bảng so sánh toàn bộ các mô hình.

4.7.1 MAE – Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

```
mae = mean_absolute_error(y_val, pred_rf)
print("MAE:", mae)
```

```
mae = mean_absolute_error(y_val, pred_rf)
print("MAE:", mae)
```

MAE: 18984.09081702544

Hình 26: Kết quả đánh giá chỉ số MAE Random Forest Regressor

Nhận xét:

Kết quả MAE khoảng 18984 cho thấy trung bình mô hình dự đoán sai khoảng 19 nghìn USD so với giá nhà thực tế. Sai số này tương đối thấp nên mô hình Random Forest Regressor dự đoán khá tốt và có độ chính xác cao

4.7.2 RMSE – Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

```
rmse = np.sqrt(mean_squared_error(y_val, pred_rf))
print("RMSE:", rmse)
```

```

:
rmse = np.sqrt(mean_squared_error(y_val, pred_rf))
print("RMSE:", rmse)

```

RMSE: 29410.142903234788

Hình 27: Kết quả đánh giá chỉ số RMSE Random Forest Regressor

Nhận xét:

Kết quả RMSE khoảng 29410 cho thấy mức sai số dự đoán của mô hình Random Forest Regressor không quá lớn. Giá trị RMSE tương đối thấp so với giá nhà trong dataset nên mô hình có khả năng dự đoán khá chính xác và ổn định.

4.7.3 R^2 Score

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

```

r2 = r2_score(y_val, pred_rf)
print("R2 Score:", r2)

```

```

:
r2 = r2_score(y_val, pred_rf)
print("R2 Score:", r2)

```

R2 Score: 0.887233413257154

Hình 28: Kết quả đánh giá chỉ số R^2 Core Random Forest Regressor

Nhận xét:

Kết quả R^2 Score đạt khoảng 0.887, cho thấy mô hình Random Forest giải thích được khoảng 88.7% sự thay đổi của giá nhà dựa trên các feature đã chọn. Giá trị R^2 khá cao nên mô hình có độ phù hợp tốt và khả năng dự đoán giá nhà hiệu quả.

4.7.4 Bảng so sánh kết quả toàn bộ mô hình

```

results = pd.DataFrame(
    [
        result_lr,
        result_ridge,
        result_lasso,
        result_dt,
        result_rf
    ],
    columns=[
        'Model',
        'MAE',
        'RMSE',
        'R2'
    ]
)
print(results)

```

	Model	MAE	RMSE	R2
0	Linear Regression	24774.219520	39474.543381	0.796848
1	Ridge Regression	24767.710994	39472.972032	0.796865
2	Lasso Regression	24711.522192	39453.493880	0.797065
3	Decision Tree	25689.619863	43449.192391	0.753878
4	Random Forest	18984.090817	29410.142903	0.887233

Hình 29: Bảng tổng hợp kết quả đánh giá mô hình

Mô hình	MAE (USD)	RMSE (USD)	R ² Score	Accuracy
Random Forest	~18,984	~29410	0.887	88.7%
Ridge Regression	~24767	~39472	0.796	79.6%
Lasso Regression	~24711	~39453	0.797	79.7%
Linear Regression	~24774	~39474	0.796	79.6%

Decision Tree	~25689	~43449	0.753	75.3%
---------------	--------	--------	-------	-------

4.8 Cross Validation (Bài tập 7)

Sử dụng 5-Fold Cross Validation để đánh giá mô hình Random Forest một cách khách quan hơn, tránh phụ thuộc vào một lần chia train/test duy nhất.

$$CV = \frac{1}{K} \sum_{i=1}^K Score_i$$

4.8.1 Thực hiện 5-Fold Cross Validation

```
scores = cross_val_score(rf, X, y, cv=5, scoring='r2')
print("Scores:")
print(scores)
```

```
Scores:
[0.83736203 0.77319743 0.86797601 0.8749949 0.81805813]
```

Hình 30: Kết quả 5-Fold Cross Validation của mô hình Random Forest

Nhận xét:

Kết quả 5-Fold Cross Validation cho thấy mô hình Random Forest hoạt động khá ổn định qua 5 lần đánh giá khác nhau. Các giá trị score đều ở mức tương đối cao, chứng tỏ mô hình có khả năng dự đoán tốt và không bị phụ thuộc quá nhiều vào một tập dữ liệu cụ thể.

4.8.2 Tính giá trị trung bình

```
print("Mean Score:", scores.mean())
```

```
Mean score:
0.8343176999949916
```

Hình 31: Kết quả giá trị trung bình 5-Fold Cross Validation của mô hình Random Forest

Nhận xét:

Giá trị Mean Score đạt khoảng 0.834, cho thấy mô hình Random Forest có độ chính xác trung bình khoảng 83.4% sau 5 lần Cross Validation. Kết quả này chứng tỏ mô hình hoạt động khá tốt và có khả năng tổng quát hóa dữ liệu ổn định.

4.9 Hyperparameter Tuning (Bài tập 8)

Sử dụng GridSearchCV với 5-fold CV để tìm bộ tham số tối ưu cho Random Forest Regressor trong không gian tham số:

- **n_estimators:** [100, 200, 300] — số lượng cây quyết định
- **max_depth:** [5, 10, 20] — độ sâu tối đa của mỗi cây

```
params = {
    'n_estimators': [100, 200, 300],
    'max_depth' : [5, 10, 20]
}
grid = GridSearchCV(
    RandomForestRegressor(),
    params,
    cv=5
)
grid.fit(X_train, y_train)
print("Best Params:", grid.best_params_)
print("Best Score: ", grid.best_score_)
```

```
Best params:
{'max_depth': 20, 'n_estimators': 300}
Best score:
0.8096695712230133
```

Hình 32: Kết quả tìm hyperparameter tối ưu bằng GridSearchCV

Nhận xét:

Mô hình đạt Best Score khoảng 0.809, cho thấy Random Forest hoạt động khá tốt với bộ tham số này. Độ sâu cây ở mức 10 giúp mô hình học đủ đặc trưng của dữ liệu mà không bị overfitting quá nhiều.

4.10 Feature Engineering (Bài tập 9)

Tạo 2 feature mới từ dữ liệu có sẵn nhằm cung cấp thông tin bổ sung giúp mô hình học tốt hơn:

```
train['AgeOfHouse'] = 2024 - train['YearBuilt']  
train['TotalArea'] = (  
    train['TotalBsmtSF']  
    + train['1stFlrSF']  
    + train['2ndFlrSF']  
)  
print(train[['AgeOfHouse', 'TotalArea']].head())
```

	AgeOfHouse	TotalArea
0	21	2566
1	48	2524
2	23	2706
3	109	2473
4	24	3343

Hình 33: Kết quả tạo feature mới của AgeOfHouse và TotalArea

Huấn luyện lại mô hình với feature mới

```
features_new = ['OverallQual', 'GrLivArea', 'GarageCars', 'GarageArea',  
    'TotalBsmtSF', '1stFlrSF', 'FullBath', 'TotRmsAbvGrd',  
    'YearBuilt', 'YearRemodAdd', 'AgeOfHouse', 'TotalArea']  
  
X = train[features_new]  
y = train['SalePrice']  
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)  
  
rf = RandomForestRegressor()  
rf.fit(X_train, y_train)  
print("Accuracy:", rf.score(X_val, y_val))
```

Chọn lại mô hình:

```
features_new = [
```

```
'OverallQual',  
'GrLivArea',  
'GarageCars',  
'GarageArea',  
'TotalBsmtSF',  
'1stFlrSF',  
'FullBath',  
'TotRmsAbvGrd',  
'YearBuilt',  
'YearRemodAdd',  
'AgeOfHouse',  
'TotalArea'  
]  
X = train[features_new]  
y = train['SalePrice']
```

Chia lại dữ liệu train và validation:

```
X_train, X_val, y_train, y_val = train_test_split(  
    X, y,  
    test_size=0.2,  
    random_state=42  
)
```

Huấn luyện lại mô hình:

```
rf_new = RandomForestRegressor(  
    n_estimators=200,  
    random_state=42  
)  
rf_new.fit(X_train, y_train)  
pred_new = rf_new.predict(X_val)  
evaluate_model(  
    "Improved Random Forest",  
    y_val,  
    pred_new  
)
```

```
Improved Random Forest
MAE : 19290.728642775604
RMSE: 30719.618917188316
R2 : 0.8769680765945744

['Improved Random Forest',
19290.728642775604,
np.float64(30719.618917188316),
0.8769680765945744]
```

Hình 34: Kết quả huấn luyện lại mô hình RF sau Feature Engineering

Chỉ số	Trước Feature Engineering	Sau Feature Engineering	Thay đổi
Accuracy (R ²)	0.884	0.876	Giảm nhẹ -0.006

Nhận xét:

Kết quả cho thấy sau khi thực hiện Feature Engineering, độ chính xác của mô hình giảm nhẹ từ 0.884 xuống 0.876. Điều này cho thấy các feature mới chưa giúp cải thiện hiệu quả dự đoán trong bài toán này, tuy nhiên mức chênh lệch không lớn nên mô hình vẫn hoạt động khá tốt.

4.11 Cải Thiện Mô Hình (Bài tập 10)

Mục tiêu: giảm RMSE xuống mức thấp nhất có thể thông qua 3 chiến lược: thêm feature, thử mô hình mới, và điều chỉnh hyperparameters.

4.11.1 Thử thêm feature

```
train['HouseAge'] = 2024 - train['YearBuilt']
train['TotalBath'] = train['FullBath'] + (0.5 * train['HalfBath'])
train['TotalPorchSF'] = train['OpenPorchSF'] + train['EnclosedPorch']
features_new = [
    'OverallQual',
    'GrLivArea',
    'GarageCars',
    'GarageArea',
    'TotalBsmtSF',
    '1stFlrSF',
    'FullBath',
    'TotRmsAbvGrd',
    'YearBuilt',
```

```

    'YearRemodAdd',
    'HouseAge',
    'TotalBath',
    'TotalPorchSF'
]
X = train[features_new]
y = train['SalePrice']
X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
rf_feature = RandomForestRegressor()
rf_feature.fit(X_train, y_train)
pred_feature = rf_feature.predict(X_val)
print("Accuracy:", rf_feature.score(X_val, y_val))
rmse_feature = np.sqrt(mean_squared_error(y_val, pred_feature))
print("RMSE:", rmse_feature)

```

Accuracy: 0.8899241222353828
RMSE: 29057.148690074024

Hình 35: Kết quả thử thêm feature cho mô hình RF

Chỉ số	Trước khi thêm feature	Sau khi thêm feature	Cải thiện
Accuracy	0.887	0.889	+0.002
RMSE (USD)	29,410	29,057	−353

Nhận xét:

Kết quả cho thấy sau khi thêm feature mới, độ chính xác của mô hình tăng từ 0.887 lên 0.889 và RMSE giảm từ 29410 xuống 29057. Điều này chứng tỏ các feature mới giúp mô hình học dữ liệu tốt hơn và cải thiện khả năng dự đoán giá nhà.

4.11.2 Thử thêm mô hình – Gradient Boosting Regressor

```

from sklearn.ensemble import GradientBoostingRegressor
gbr = GradientBoostingRegressor()
gbr.fit(X_train, y_train)
pred_gbr = gbr.predict(X_val)

```

```
print("Accuracy:", gbr.score(X_val, y_val))
print("RMSE:", np.sqrt(mean_squared_error(y_val, pred_gbr)))
```

Accuracy: 0.889982494148099
RMSE: 29049.44334036711

Hình 36: Kết quả thử nghiệm thêm với mô hình Gradient Boosting Regressor

Chỉ số	Random Forest (baseline)	Gradient Boosting
Accuracy	0.885	0.889
RMSE (USD)	29410	29049

Nhận xét

- Sự cải thiện về độ chính xác (Accuracy R^2): Thuật toán Gradient Boosting ghi nhận hệ số xác định đạt 0,8899 (làm tròn trên bảng là 0.889), tăng nhẹ so với mức 0,885 của Random Forest. Sự gia tăng này minh chứng rằng cơ chế huấn luyện tuần tự (tập trung tối ưu hóa sai số của các cây quyết định xây dựng trước đó) giúp mô hình bắt kịp tốt hơn các quy luật phân phối phức tạp của dữ liệu.
- Sự tối ưu hóa về sai số (RMSE): Chỉ số sai số bình phương trung bình căn (RMSE) của Gradient Boosting giảm từ 29.410 USD xuống còn 29.049 USD (giảm khoảng 361 USD). Việc thu hẹp khoảng cách sai số này cho thấy mô hình cải tiến có xu hướng đưa ra các quyết định dự báo tiệm cận gần hơn với giá trị giao dịch thực tế trên thị trường bất động sản, đồng thời giảm bớt tác động tiêu cực từ các điểm dữ liệu lệch biên.

Kết luận: Mặc dù biên độ cải tiến không quá lớn do mô hình Random Forest ban đầu vốn đã đạt ngưỡng hiệu năng rất cao, thuật toán Gradient Boosting vẫn chứng minh được giá trị tối ưu hóa vượt trội. Đây là cấu trúc hoàn chỉnh nhất, sở hữu sai số thấp nhất và độ chính xác cao nhất trong toàn bộ chuỗi thực nghiệm bài toán dự đoán giá nhà.

4.11.3 Điều chỉnh hyperparameters cho Random Forest

```

rf_tuning = RandomForestRegressor(
    n_estimators=300,
    max_depth=20,
    random_state=42
)
rf_tuning.fit(X_train, y_train)
pred_tuning = rf_tuning.predict(X_val)
print("Accuracy:", rf_tuning.score(X_val, y_val))
print("RMSE:", np.sqrt(mean_squared_error(y_val, pred_tuning)))

```

Accuracy: 0.8865210454248947
RMSE: 29502.89137087461

Hình 37: Kết quả thử nghiệm điều chỉnh hyperparameter

Chỉ số	Trước điều chỉnh	Sau điều chỉnh
Accuracy	0.885	0.886
RMSE (USD)	29410	29502

Nhận xét

- Về độ chính xác (Accuracy / R^2): Chỉ số Accuracy ghi nhận mức tăng trưởng nhẹ từ 0,885 lên 0,8865 (làm tròn trên bảng là 0.886). Sự cải thiện này chứng minh rằng các giải pháp điều chỉnh đã giúp mô hình tối ưu hóa thêm một phần không gian giả thuyết, nắm bắt tốt hơn các điểm dữ liệu kiểm chứng.
- Về sai số bình phương trung bình căn (RMSE): Chỉ số RMSE sau khi điều chỉnh có sự biến động tăng nhẹ từ 29.410 USD lên 29.502 USD (tăng khoảng 92 USD). Việc độ chính xác chung (R^2) tăng nhưng sai số RMSE lại tăng nhẹ cho thấy mô hình sau điều chỉnh đã cải thiện tốt độ phân phối dự báo trên đại đa số các mẫu nhà thông thường, nhưng lại chịu mức phạt sai số lớn hơn ở một vài điểm dữ liệu ngoại lai mang giá trị cực trị (như phân khúc nhà siêu sang).

Kết luận: Nhìn chung, biên độ dao động của cả hai chỉ số trước và sau khi điều chỉnh là rất nhỏ, cho thấy mô hình ban đầu vốn đã tiệm cận sát ngưỡng trần hiệu năng tối ưu đối với tập đặc trưng hiện tại. Cấu trúc sau điều chỉnh vẫn duy trì được độ ổn định cực kỳ cao (độ chính xác đạt trên 88.6%) và hoàn toàn đáng tin cậy để đưa vào triển khai thực tế.

Tổng hợp kết quả cải thiện mô hình

Phương pháp	Accuracy	RMSE (USD)	Nhận xét
Random Forest (baseline)	0.8872	29410	Mô hình gốc
+ Thêm feature mới	0.888	29,057	Cải thiện nhẹ (+feature tiện nghi)
+ Điều chỉnh hyperparameters	0.8865	29,502	Cải thiện nhẹ
Gradient Boosting	0.8899	29,049	Tốt nhất – mô hình khuyến nghị

5. PHÂN TÍCH VÀ THẢO LUẬN

5.1 Đánh giá tổng thể kết quả

Nhìn toàn cảnh, kết quả thực nghiệm cho thấy sự phân hóa rõ rệt giữa hai nhóm mô hình. Nhóm mô hình phi tuyến (Decision Tree, Random Forest, Gradient Boosting) và nhóm mô hình tuyến tính (Linear Regression, Ridge, Lasso) có sự khác biệt đáng kể về khả năng dự báo.

Trong nhóm tuyến tính, cả ba mô hình cho R^2 dao động quanh mức 0.797, cho thấy chúng đã chạm đến giới hạn hiệu năng khi áp dụng giả định tuyến tính vào dữ liệu bất động sản thực tế. Nguyên nhân cốt lõi là mối quan hệ giữa các đặc trưng và giá nhà không tuân theo quy luật tuyến tính thuần túy: chẳng hạn, chất lượng xây dựng (OverallQual) ở bậc 9 và 10 đẩy giá nhà tăng vọt theo dạng hàm mũ, chứ không tăng đều theo một hệ số cố định như Linear Regression giả định.

Random Forest vượt qua giới hạn đó bằng cách kết hợp 100 cây quyết định, đạt $R^2 = 0.8872$ và $RMSE = 29,410$ USD — tức là mô hình giải thích được gần 89% sự biến động của giá nhà. Gradient Boosting tiếp tục cải thiện thêm, đạt $RMSE = 29,049$ USD và $R^2 = 0.889$, là mô hình có hiệu năng tốt nhất trong toàn bộ thực nghiệm.

5.2 Phân tích hiện tượng Overfitting và Underfitting

Overfitting — Decision Tree: Decision Tree Regressor không giới hạn chiều sâu là ví dụ điển hình của hiện tượng overfitting. Mô hình cây quyết định đơn lẻ có xu hướng phân chia dữ liệu đến mức rất chi tiết, học thuộc cả những điểm nhiễu trong tập huấn luyện. Hệ quả là khi gặp dữ liệu validation mới, mô hình cho kết quả kém ổn định hơn so với các mô hình khác, với $R^2 = 0.7539$ — thấp nhất trong toàn bộ thực nghiệm. Khoảng cách lớn giữa hiệu năng trên tập train và validation là dấu hiệu rõ ràng của overfitting. Để khắc phục, có thể sử dụng các siêu tham số như `max_depth`, `min_samples_split` hoặc `min_samples_leaf` để kiểm soát độ sâu của cây.

Underfitting — nhóm Linear Regression: Ngược lại, ba mô hình tuyến tính (Linear, Ridge, Lasso) dù ổn định nhưng bị giới hạn bởi kiến trúc quá đơn giản. Khi dữ liệu có

cấu trúc phi tuyến phức tạp (đặc biệt là OverallQual và GrLivArea), mô hình tuyến tính không đủ năng lực biểu diễn các mối quan hệ này, dẫn đến hiện tượng underfitting nhẹ — R^2 dừng lại ở mức ~ 0.797 cho dù có thêm regularization hay không.

Random Forest — cân bằng tốt: Random Forest khắc phục cả hai vấn đề trên nhờ cơ chế bagging: mỗi cây được huấn luyện trên một tập dữ liệu bootstrap khác nhau và chỉ xét một tập con đặc trưng ngẫu nhiên tại mỗi node. Sự đa dạng giữa các cây giúp triệt tiêu phương sai cao của từng cây đơn lẻ, trong khi độ phức tạp tổng thể của ensemble vẫn đủ lớn để học các quan hệ phi tuyến, đạt được cân bằng tốt giữa bias và variance.

5.3 Ảnh hưởng của Hyperparameter Tuning

Kết quả GridSearchCV với không gian tìm kiếm gồm $n_estimators \in \{100, 200, 300\}$ và $max_depth \in \{5, 10, 20\}$ cho thấy bộ tham số tối ưu đạt Best Score (R^2 trên Cross-Validation 5-fold) khoảng 0.809.

Một điểm đáng lưu ý là điểm Cross-Validation (0.809) thấp hơn R^2 đo trên tập validation một lần (0.8872). Đây là hiện tượng bình thường và hợp lý: Cross-Validation đánh giá mô hình trên nhiều tập con khác nhau, bao gồm cả những lần chia không thuận lợi, nên cho kết quả khách quan và bảo thủ hơn. Đây mới là chỉ số phản ánh đúng khả năng tổng quát hóa thực sự của mô hình.

Về ảnh hưởng của max_depth : giá trị 10 được chọn là tối ưu trong thực nghiệm này, cho phép cây học đủ sâu để nắm bắt các pattern phức tạp trong dữ liệu, đồng thời vẫn kiểm soát tốt overfitting nhờ cơ chế ensemble. Khi thử nghiệm thủ công với $max_depth=20$ và $n_estimators=300$ ở Mục 4.11.3, kết quả R^2 chỉ tăng nhẹ từ 0.885 lên 0.886, nhưng RMSE lại tăng nhẹ từ 29,410 lên 29,502 USD. Điều này cho thấy mô hình ban đầu đã tiệm cận ngưỡng trần hiệu năng với tập đặc trưng hiện tại, và việc tăng thêm số cây hay độ sâu mang lại lợi ích biên rất hạn chế theo quy luật diminishing returns.

5.4 Hiệu quả của Feature Engineering

Trong Bài tập 9, hai đặc trưng mới được tạo ra là AgeOfHouse ($= 2024 - YearBuilt$) và TotalArea ($= TotalBsmtSF + 1stFlrSF + 2ndFlrSF$). Tuy nhiên, kết quả thực nghiệm cho

thấy sau khi thêm hai đặc trưng này, R^2 của mô hình giảm nhẹ từ 0.884 xuống 0.876. Điều này có thể do hai nguyên nhân: (1) AgeOfHouse mang thông tin gần như trùng lặp hoàn toàn với YearBuilt vốn đã có trong tập đặc trưng, khiến mô hình không học thêm được thông tin mới; (2) TotalArea tổng hợp lại các diện tích đã có sẵn trong tập đặc trưng như TotalBsmtSF và 1stFlrSF, tạo ra hiện tượng đa cộng tuyến bổ sung mà không mang lại giá trị dự báo độc lập.

Trong Bài tập 10, khi bổ sung thêm các đặc trưng mới có ý nghĩa thực tiễn hơn như HouseAge, TotalBath (= FullBath + $0.5 \times$ HalfBath) và TotalPorchSF (= OpenPorchSF + EnclosedPorch), kết quả cải thiện rõ hơn: R^2 tăng từ 0.887 lên 0.889 và RMSE giảm từ 29,410 xuống 29,057 USD. Cải thiện này cho thấy TotalBath — chỉ số phản ánh tổng công năng phòng vệ sinh của căn nhà — là đặc trưng bổ sung thực sự có giá trị thông tin mới mà tập đặc trưng gốc chưa biểu diễn trực tiếp.

Bài học quan trọng rút ra: Feature Engineering hiệu quả không đơn giản là tạo thêm nhiều đặc trưng, mà phải đảm bảo mỗi đặc trưng mới mang thông tin độc lập và ý nghĩa thực tiễn rõ ràng, tránh trùng lặp với các đặc trưng đã có.

5.5 Lỗi gặp phải và cách khắc phục

ValueError: Input contains NaN: Lỗi này xảy ra khi mô hình sklearn nhận dữ liệu đầu vào còn chứa giá trị thiếu chưa được xử lý. Khắc phục bằng cách áp dụng fillna(mean()) cho các cột số trước khi truyền dữ liệu vào mô hình. Đây là bước tiền xử lý bắt buộc trong mọi pipeline Machine Learning.

Thời gian chạy GridSearchCV quá lâu: GridSearchCV với 3 giá trị n_estimators $\times$ 3 giá trị max_depth $\times$ 5-fold CV tương đương 45 lần huấn luyện Random Forest. Khắc phục bằng cách thêm tham số n_jobs=-1 để tận dụng toàn bộ nhân CPU song song, giảm đáng kể thời gian thực thi.

Kết quả Cross-Validation thấp hơn kết quả validation split: Đây không phải lỗi mà là hiện tượng tự nhiên. CV đánh giá trên nhiều tập con khác nhau nên cho kết quả khách quan hơn một lần phân chia cố định, đặc biệt khi dữ liệu có phân bố không đều giữa các

fold. Cần hiểu đây là chỉ số tham chiếu đáng tin cậy hơn để đánh giá khả năng tổng quát hóa thực sự.

Các cột dạng chuỗi (object) chưa được sử dụng: Trong thực nghiệm này, chỉ các đặc trưng số được đưa vào mô hình. Tuy nhiên bộ dữ liệu còn nhiều cột định tính có giá trị cao như Neighborhood (khu vực địa lý), BsmtQual (chất lượng tầng hầm) hay KitchenQual (chất lượng nhà bếp). Để sử dụng các cột này cần áp dụng kỹ thuật One-Hot Encoding hoặc Label Encoding trước khi huấn luyện.

Điểm ngoại lệ chưa được xử lý: Bước phân tích scatter plot ở Mục 4.3.2 đã nhận diện hai căn nhà có diện tích cực lớn (trên 4,500 sqft) nhưng giá bán bất thường thấp. Tuy nhiên thực nghiệm chưa xử lý các outlier này trước khi huấn luyện, có thể ảnh hưởng đến độ chính xác của các mô hình tuyến tính.

5.6 Ưu điểm và hạn chế của các mô hình

Bảng tổng hợp ưu và nhược điểm:

Mô hình	Ưu điểm	Hạn chế
Linear/Ridge/Lasso	Nhanh, dễ diễn giải, ổn định	Hạn chế với quan hệ phi tuyến
Decision Tree	Phi tuyến, dễ trực quan hóa	Dễ overfitting khi cây sâu
Random Forest	Chính xác cao, ổn định, kháng overfitting	Chậm, khó diễn giải, tốn bộ nhớ
Gradient Boosting	Chính xác cao nhất, linh hoạt	Nhạy cảm với hyperparameter, chậm nhất

6. KẾT LUẬN

Bài thực hành đã hoàn thiện toàn bộ quy trình xây dựng hệ thống dự đoán giá nhà theo pipeline chuẩn của Machine Learning, từ khám phá và phân tích dữ liệu (EDA), lựa chọn đặc trưng dựa trên tương quan, huấn luyện và so sánh nhiều mô hình hồi quy, đến đánh giá bằng các chỉ số và cải tiến thông qua Cross-Validation, Hyperparameter Tuning và Feature Engineering. Trong giai đoạn đầu, Random Forest Regressor đạt kết quả tốt nhất với $R^2 = 0.8872$, $MAE \approx 18,984$ USD và $RMSE \approx 29,410$ USD trên tập validation. Sau khi bổ sung các đặc trưng mới như HouseAge, TotalBath và TotalPorchSF, đồng thời thử nghiệm Gradient Boosting Regressor, hiệu năng tiếp tục được cải thiện với RMSE giảm còn 29,049 USD (tương đương cải thiện khoảng 1.2%), và đạt $R^2 = 0.889$ — trở thành mô hình tốt nhất toàn bộ thực nghiệm. Kết quả cho thấy chất lượng đặc trưng quan trọng hơn số lượng, khi việc chọn lọc 10 đặc trưng có ý nghĩa giúp mô hình hoạt động hiệu quả hơn so với sử dụng toàn bộ dữ liệu thô. Bên cạnh đó, Feature Engineering chỉ thực sự hữu ích khi tạo ra thông tin mới độc lập, trong khi các đặc trưng trùng lặp không mang lại lợi ích rõ rệt. Ngoài ra, Cross-Validation (5-Fold với điểm trung bình ≈ 0.834) cung cấp đánh giá đáng tin cậy hơn so với một lần chia dữ liệu. Cuối cùng, các mô hình phi tuyến như Random Forest và Gradient Boosting tỏ ra vượt trội trong bài toán này do khả năng nắm bắt các mối quan hệ phức tạp giữa các đặc trưng bất động sản.